SPRING CLOUD · MICROSERVICES IN ACTION

Spring Cloud

微服务实战 · 一次请求的分层之旅

不加一行配置地讲清：

一次调用怎么穿过 Nginx、网关、Feign、Hystrix，

又为什么"配了不生效"、"一慢全崩"

基于 Spring Cloud + Nginx · 695 页原著 + 502 个源码文件读薄

目　录

01 序：为什么读这本 Spring Cloud 3

02 一、一次请求的链路速查 4

03 二、12 章一句话速查 4

04 三、症状 → 章节 5

05 四、20 个自测问题 6

06 五、一个提醒 6

07 第1章　全局地图：一次请求穿过哪些组件 7

08 第2章　服务注册与发现：Eureka 的 AP 取舍 10

09 第3章　配置中心与版本适配：为什么"配了不生效" 12

10 第4章　客户端负载均衡：Ribbon 与 Feign 各管一段 15

11 第5章　Feign 动态代理：一个接口怎么变成一次 HTTP 调用 18

12 第6章　RPC 保护：舱壁、熔断器与滑动窗口 21

13 第7章　响应式与背压：RxJava 在调用链里的位置 24

14 第8章　网关：Zuul 过滤器链与路由边界 29

15 第9章　身份识别：Spring Security、JWT 与会话共享 34

16 第10章　Nginx：进程模型、location 与反向代理 39

17 第11章　限流：计数器、漏桶、令牌桶与分布式实现 42

18 第12章　秒杀全链路：把前 11 章串成一条线 44

19 结语：微服务不是组件的堆砌 46

20 把书读薄：一张判断清单 46

21 附录：三张速查表 47

22 最后一句 48

序：为什么读这本 Spring Cloud

会用 Spring Cloud 和懂 Spring Cloud 之间，隔着一整层"分层"。

加个 `@EnableDiscoveryClient` 就能注册，加个 `@FeignClient` 就能调远程接口，`timeout`、`retries`、`fallback` 配一配就能上线。可一旦出问题——"服务实例已经停了，消费端还在往它发请求""明明配了 3 秒超时，链路还是卡 30 秒""限流配了 1000 QPS，压测照样把数据库打穿""秒杀超卖，库存扣成了负数"——配置项就不管用了，你得知道这些参数最终落在了哪个组件的哪一层、在哪个时刻被读取、被谁覆盖。

这本书基于《Spring Cloud、Nginx 高并发核心编程》（695 页）和它的配套开源脚手架（502 个 Java 文件）写成，**不是原书摘要**。全书 12 章，按"一次请求从浏览器到数据库，依次穿过哪些组件"的顺序组织：

部分	章节	回答的问题
骨架	1–3	一次请求经过哪些组件？注册中心重 AP 意味着什么？为什么"配了不生效"？
调用链	4–6	负载均衡、动态代理、熔断各自管哪一段，为什么不能互相替代？
网关与安全	7–9	响应式背压是什么？网关过滤器链怎么走？认证和会话共享怎么落地？
边缘与高并发	10–12	Nginx 为什么能扛并发？限流三种算法差在哪？秒杀怎么把它们串成一条线？

> 读法建议：第 1 章是地图，务必先读。第 2、3 章是后面所有远程调用和配置问题的前置——不理解"服务发现是 AP 的、配置有加载阶段之分"，就理解不了调用失败和配置不生效。其余各章自包含，可以按需跳读，每章都有结构表、示意图和"容易误解的地方"。
>
> **版本口径**：原书基于 Spring Cloud Netflix 体系（Eureka / Ribbon / Hystrix / Zuul / Config）。这套组件后来陆续进入维护模式，工程上已有替代：Nacos / Consul、Spring Cloud LoadBalancer、Resilience4j / Sentinel、Spring Cloud Gateway、Nacos Config。本书以原书**机制**为主体（代理、熔断、滑动窗口、反向代理、限流算法的原理没有过时），并在每章用"今天怎么做"给出**现代替代组件的映射**——变的只是实现，不变的职责边界。

一、一次请求的链路速查



每一层只改自己那段：Nginx 改连接与静态流量，网关改服务级路由与横切，Feign 改"请求长什么样"，Ribbon 改"发给谁"，Hystrix 改"出问题时怎么办"，注册/配置中心改"实例与配置从哪来"。把 A 层的职责塞进 B 层（例如"在网关里写业务""在应用里做限流"），就是大多数微服务事故的开端。

二、12 章一句话速查

章	一句话	什么时候翻它
1 全局地图	一次请求穿过 Nginx→网关→消费者→注册/配置→提供者，每层只改自己那段	刚接手微服务，想建立整体印象
2 服务注册与发现	Eureka 重 AP，下线实例靠心跳剔除+本地缓存，所以"已停服还在被调用"	服务上下线抖动、调用到已下线实例、发现延迟
3 配置中心与版本适配	配置有加载阶段之分，配了不生效先问"被谁覆盖、何时被读"	改配置要重启才生效、@RefreshScope 不刷新、版本冲突
4 客户端负载均衡	Ribbon 选机器、Feign 拼请求、Hystrix 兜底，三者串联各管一段	负载不均、重试导致重复下单、服务端/客户端 LB 混淆

章	一句话	什么时候翻它
5 Feign 动态代理	一个接口被 JDK 代理变成 HTTP 调用，三套超时谁小谁说了算	超时配了不生效、想统一加鉴权头、理解调用栈
6 RPC 保护	舱壁隔离线程、熔断断失败调用、滑动窗口统计，挡的是雪崩传导	一个下游慢拖垮整条链路、降级里又调远程
7 响应式与背压	观察者模式把"拉"变"推"，背压是内部速率协商不是入口限流	异步编排、Hystrix 执行模型、高并发事件流
8 网关	Zuul 四型过滤器按 pre/route/post/error 串成横切链，别在网关写业务	统一鉴权/日志/限流放哪、网关成为单点
9 身份识别	认证辨人、鉴权定权，JWT 无状态但难主动失效，网关验签后向下透传	微服务间会话共享、令牌刷新、内部调用信任边界
10 Nginx	master/worker 异步非阻塞扛并发，location 匹配优先级与 proxy_pass 拼路径是高频坑	Nginx 调优、rewrite 循环、反向代理路径对不上
11 限流	计数器/漏桶/令牌桶三种算法，分布式限流要靠原子计数，不是每实例各自限	压测打穿数据库、限流形同虚设、阈值怎么定
12 秒杀全链路	就近拦截、动静分离、原子扣库存、异步削峰，把前 11 章拼成一条线	超卖、重复下单、峰值流量整形、全链路容量规划

三、症状 → 章节

现象	先看
服务实例已经停了，调用方还一直报错 / 打到已下线实例	第 2 章、第 4 章
改了配置要重启所有实例才生效	第 3 章
配了 timeout 还是整条链路卡死	第 5 章、第 6 章
一个下游慢，整条链路像多米诺一样雪崩	第 6 章
网关里逻辑越写越多，改一处影响全部流量	第 8 章
登录态在微服务之间丢、会话不同步	第 9 章
Nginx 配置改了不生效、proxy_pass 路径对不上	第 10 章
压测打穿数据库、限流形同虚设	第 11 章
秒杀超卖、重复下单	第 12 章

现象	先看
接口偶发超时又自愈	第 2 章（缓存）、第 6 章（熔断）

四、20 个自测问题

1. 服务端负载均衡（Nginx）和客户端负载均衡（Ribbon）的本质区别是什么？
2. Eureka 重 AP，这意味着注册中心不可用时，已经拿到实例列表的消费方还能不能调服务？
3. 服务进程被 `kill -9` 强杀，注册中心要过多久才把实例剔除？期间请求会怎样？
4. 为什么"服务下线了，还有请求打过去"？根本原因是哪一层？
5. 配置中心里，bootstrap 阶段加载的配置和 application 阶段加载的配置，生效范围有什么不同？
6. 改了一个配置但所有实例都要重启才生效，最可能在哪一环出了问题？
7. Feign 的 `readTimeout` 和 Ribbon 的超时、Hystrix 的超时，三者大小关系应该怎么排？反过来会怎样？
8. 客户端重试打在"创建订单"这种写接口上，会带来什么后果？怎么避免？
9. JDK 动态代理为什么要求目标必须是接口？Feign 客户端为什么一定是接口？
10. Hystrix 的线程池隔离和信号量隔离，分别适合什么场景？代价各是什么？
11. 熔断器从 Open 回到 Half-Open 需要什么条件？"错误率超过阈值"的"错误率"是在哪个时间窗里统计的？
12. 滑动窗口相比一个全局计数器，解决了限流的什么问题？
13. 观察者模式和迭代器模式，在"谁控制节奏"这一点上有什么区别？
14. 背压（backpressure）和限流（rate limiting）是一回事吗？分别发生在链路的哪一段？
15. Zuul 的 pre / route / post / error 四类过滤器，各自在什么时候运行？一个异常会在哪类过滤器被统一处理？
16. 认证（authentication）和鉴权（authorization）的分界在哪？网关做认证之后，下游服务还要不要再鉴权？
17. JWT 相比 session，为什么"无法主动失效"？这会带来什么工程上的代价？
18. Nginx 的 `location` 匹配优先级顺序是什么？为什么"规则没生效"常是优先级问题？
19. 漏桶算法和令牌桶算法，谁允许突发流量？谁的"输出速率"是恒定的？
20. 分布式限流为什么不能简单"每台实例限 N QPS"？要保证全局限额需要什么原语？

五、一个提醒

这本书讲机制，不替代官方文档，也不替代你读源码。12 章之后你会建立一张"组件→职责→配置落点"的地图，但真正的内化发生在你把自己项目的某一个超时、某一次雪崩、某一条 Nginx 配置对上号的那一刻。读的时候带着你线上那个具体的问题，会比通读更有用。

第1章　全局地图：一次请求穿过哪些组件

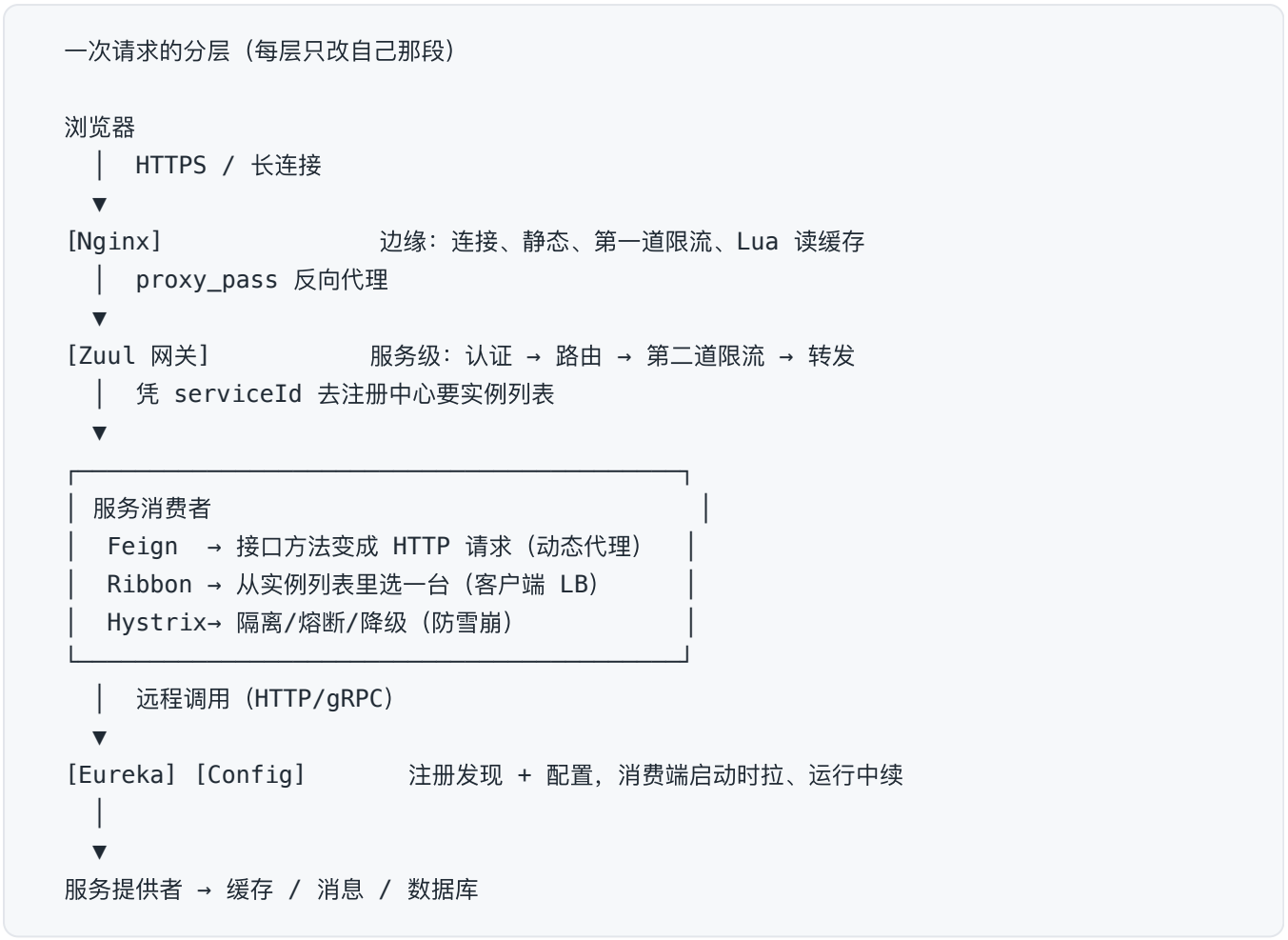
一句话：一次外网请求会依次穿过 Nginx、网关、消费者、注册/配置中心、提供者，每一层只改自己那段，职责错位就是事故源头。

为什么需要它：先看四个现场。现场一：你给接口配了 3 秒超时，压测却整体卡 30 秒——问题不在你的业务逻辑，而在某层超时配置没生效。现场二：下游一个服务慢，结果整条调用链像多米诺一样雪崩——因为你没有在"上游调用下游"的那一层做隔离。现场三：限流配了 1000 QPS，压测照样把数据库打穿——限流放错了层，离用户太远，拒绝成本太高。现场四：Nginx 改了一条 `proxy_pass` 路径，结果路由全错——你改的是最边缘的一层，却影响整条链。四个现场的共同点：你不知道一次请求**依次经过哪些组件**、每层**只改什么**、哪一层**该为这次失败负责**。这一章把这张地图立起来。

核心结构：

组件	角色	只负责	不负责
Nginx / OpenResty	边缘反向代理	连接复用、静态流量、第一道限流、Lua 边缘逻辑	服务发现、业务路由
Zuul 网关	内部网关	认证、路由、横切日志/限流	具体业务逻辑
服务消费者	调用方	Feign 拼请求、Ribbon 选实例、Hystrix 隔离熔断	服务注册、配置存储
注册中心 Eureka	服务目录	实例注册、心跳剔除、发现	调用、负载均衡策略
配置中心 Config	配置仓库	集中配置、动态刷新	业务编排
服务提供者	业务实现	处理请求、落库	知道谁在调自己

关键设计：



关键分界：**Nginx 与网关之间是"边缘流量 vs 服务流量"的边界；消费者内部 Feign/Ribbon/Hystrix 是"拼请求 / 选机器 / 兜底"的三段职责**。任何一层去干另一层的活（在网关写业务、在应用里做边缘限流），都会同时失去自己的能力和对方的可靠性。

拆解：

1.1 Spring Boot 与 Spring Cloud 的职责分界

单体业务服务基于 Spring Boot 应用。Spring Boot 解决"快速方便地开发一个单体服务"；Spring Cloud 解决"这些单体之间如何协调治理"——配置管理、服务发现、熔断、路由、全局锁、分布式会话。Spring Boot 可以离开 Spring Cloud 独立使用，但 Spring Cloud 依赖 Spring Boot 存在。一句话：Spring Boot 造一个个单体，Spring Cloud 把它们编排成一个系统。

1.2 为什么是这么多独立组件，而不是一个"微服务框架"

因为每一类问题性质不同：注册发现是"目录与心跳"（AP 语义），配置是"外部化与刷新"，调用保护是"资源隔离"（线程语义），网关是"横切与边界"（HTTP 语义），反向代理是"连接与静态"（OS/网络语义）。把它们拆开，每一层才能独立演进、独立失败、独立替换。代价是你要清楚每次失败归哪一层——这正是本书后面各章的主题。

1.3 CAP 在哪一层被权衡

注册中心重 AP 还是 CP，决定"网络分区时你看到的是旧实例还是拒绝服务"。原书选 Eureka（AP）：宁可返回可能过时的实例，也不让注册中心自身成为单点阻塞。配置中心、数据库则是另一套权衡。理解"哪一层容忍不一致、哪一层必须一致"，是设计微服务的第一课。

1.4 秒杀：贯穿全书的样本

原书以"秒杀"作为统一实战案例，原因很简单：它同时触发限流（第 11 章）、缓存、异步削峰、库存原子扣减、网关与边缘拦截（第 8/10 章）。后面 11 章的机制，最终在第 12 章被拼成一条链路。读每一章时，都可以想一句："这一章的机制，在秒杀链路的第几层起作用？"

容易误解的地方：

1. 很多人以为"加了 Spring Cloud 依赖，服务就自动高可用了"，其实是多了责任——每一层都可能成为新的失败点，不配置隔离/熔断，多一个依赖就多一个雪崩入口。
2. 以为"网关能解决所有横切问题"，其实网关只适合无状态、服务级的横切；把业务判定塞进网关，网关会膨胀成单点瓶颈。
3. 以为"注册中心挂了，整个系统就挂了"，AP 型注册中心挂了，已缓存实例列表的消费方仍能调用的，只是无法发现新实例。
4. 以为"限流配在应用里就行"，离用户越远，拒绝的代价越高；边缘（Nginx）就该先挡掉大部分超额流量。

记住这一句：微服务的复杂度不在组件多，而在每一层都在把"别人的失败"变成"自己的事"——把职责切干净，地图就清楚了。

第2章　服务注册与发现：Eureka 的 AP 取舍

一句话：Eureka 选择 AP——宁可返回可能过时的实例，也不让注册中心自身成为阻塞单点；代价是"已停服的实例还会被调用一小段时间"。

为什么需要它：一个微服务系统由几十上百个服务单元、每个又有众多实例组成（原书举例 Netflix 600+ 服务单元）。网状依赖下，必须有一个"注册中心"统一维护实例清单与健康状态。现场：你 `kill -9` 停掉一个实例，调用方却还在往它发请求、报连接拒绝；或者你新起一个实例，要等一会儿才被调用到。这两个现象的根因不是 bug，而是注册发现的**AP 语义 + 多级缓存**共同决定的。这一章讲清：注册中心、服务提供者、客户端组件各自做什么，心跳与剔除的时序，以及"为什么服务下线了还有请求打过去"。

核心结构：

角色	关键职责
注册中心	维护服务注册表；健康检查（定时探活）；提供实例清单查询/管理 API
服务提供者	服务注册（启动时上报）；心跳续约；健康 API
注册中心客户端	服务发现（拉清单）；实例缓存；注册/续约的交互
远程客户端（Feign 等）	用清单里的实例地址完成 RPC

关键设计：

```
心跳–续约–剔除–缓存 时序

提供者启动 ──register──▶ 注册中心（写入注册表）
     │
     ├─ 每 30s ──heartbeat──▶ 注册中心（续约，刷新 lastRenewal）
     │
注册中心每 60s 扫描：
     lastRenewal 超过 (60s×2=120s) 未续约 ──▶ 从注册表剔除
     │
消费者 ──fetch registry──▶ 拉全量/增量 ──▶ 本地缓存（每 30s 刷新）
     │
调用时：从【本地缓存】取实例，而不是每次问注册中心
     │
# 关键：消费者调用用的是【本地缓存快照】
# 实例被剔除后，缓存还要再过 30s 才刷新 → 这期间仍会打到已死实例
```

自我保护（Renewal 阈值）：如果短时间内大量实例心跳丢失（如网络分区），Eureka 会进入自我保护，不再剔除，避免把"网络抖动"误判成"大面积下线"导致可用实例被清空。

拆解：

2.1 三个角色与各自职责

注册中心是目录，负责注册表与探活；提供者是租户，负责上报与续约；客户端组件既帮提供者注册/续约，又帮消费者发现/缓存。三者通过"注册—续约—发现"三件事解耦：提供者不关心谁在调，消费者不关心提供者住哪，都由注册中心中转。

2.2 心跳续约与延迟剔除

续约让注册中心知道"我还活着"。剔除有宽限期：超过若干倍心跳周期没续约才剔除，避免单次 GC 停顿被误删。提供者恢复心跳后又能被加回。**真正的坑在消费端缓存**：消费者拿的是本地缓存快照，注册表更新后缓存要等下一个刷新周期才变。所以"实例已死但仍在被调用"的窗口 = 剔除等待 + 缓存刷新延迟，通常几十秒。这就是为什么优雅下线要"先摘流量、再停进程"。

2.3 Eureka 重 AP 的含义

CAP 里 Eureka 选 AP：网络分区时，注册中心优先保证可用（继续返回可能过时的清单），而不是阻塞等待一致性。对比 ZooKeeper 重 CP：它保证强一致，但选举期间可能拒绝服务。对"调用方宁可拿到旧实例也不要拿不到任何实例"的场景，AP 更合适。**注意**：AP 不是"不会错"，而是"错一阵子"——你得在调用层用重试 + 故障转移来消化这份过期。

2.4 多级缓存导致的"发现延迟"

注册表 → 只读缓存（ResponseCache，定时更新）→ 消费者本地缓存，层层都有 TTL。想让"新实例立刻被发现"几乎不可能，只能缩短各级刷新间隔，或在客户端做故障转移。理解这一点，就不会再把"刚启动的实例没流量"当成注册失败。

今天怎么做：Nacos / Consul 提供了 AP/CP 可切换或默认更优的语义；Nacos 同时支持服务发现与配置，健康检查更实时。但"本地缓存导致发现延迟、下线要摘流量"的判断标准不变。Kubernetes 环境下，常常用 Service + Endpoint 或 Istio 的服务发现替代独立注册中心——但 AP/CP 的权衡依然存在。

容易误解的地方：

1. 以为"注册中心挂了系统就全挂"——AP 型注册中心挂了，已缓存清单的消费方仍能调用，只是无法发现新实例/剔除死实例。
2. 以为"kill 掉进程，调用方立刻就不打了"——中间隔着剔除等待 + 客户端缓存刷新，通常还要几十秒。
3. 以为"服务发现是每次调用都查注册中心"——实际是拉到本地缓存、本地选实例，注册中心不在调用关键路径上。
4. 以为"自我保护是 bug"——它是防止网络抖动把全部实例误删的兜底，但也会让真正下线的实例赖在表里更久。

记住这一句：服务发现的 AP 语义 + 多级缓存，决定了"已停服还在被调用"是设计内行为，不是故障——靠客户端故障转移和优雅摘流来消化它。

第3章　配置中心与版本适配：为什么"配了不生效"

一句话：配置不生效，九成不是没写对，而是它"在哪个阶段被加载、被谁覆盖、在哪个时刻被读取"你没搞清楚；再加上 Spring Cloud 与 Spring Boot 的版本是硬约束。

为什么需要它：现场一：你改了配置，要重启所有实例才生效——说明这份配置没走配置中心，或被写死了。现场二：你加了 `@RefreshScope`，结果某些 bean 还是旧的——因为不是所有配置都能热刷新。现场三：项目一引入新组件就报 `ClassNotFound` 或自动配置不生效——这是 Spring Cloud 与 Spring Boot 版本不匹配。现场四：你在 `application.yml` 里配了 bootstrap 阶段才需要的项，结果根本没被读。这四个现场对应配置中心的四件事：**集中化、外部化、动态刷新、版本兼容**。这一章给出排查的确定性顺序。

核心结构：

概念	作用
配置仓库	Git/文件里的配置文件，按应用/环境/分支组织
Config Server	把仓库配置以 HTTP 接口暴露给客户端
Config Client	启动时拉取配置，支持 `@RefreshScope` 热刷新
bootstrap 阶段	框架启动最早阶段，加载"引导配置"（如配置中心地址本身）
application 阶段	业务配置加载，可被 bootstrap 的配置覆盖
版本矩阵	Spring Cloud 与 Spring Boot 有强绑定版本对应关系

关键设计：

```
配置加载的两个阶段（顺序决定"谁覆盖谁"）

┌─ bootstrap 阶段（最先）──────────────────────┐
│  读 bootstrap.yml / --spring.cloud.config.uri │
│  目的：先知道"配置中心在哪"                    │
│  这里配错 → 连不上配置中心 → 直接启动失败      │
└──────────────────────────────────────────────┘
                    │
┌─ application 阶段 ──────────────────────────┐
│  从配置中心拉业务配置，组装 Environment        │
│  同名配置：后加载的覆盖先加载的                │
│  优先级（高→低）：命令行 > 环境变量 > 配置中心 > 本地yml │
└──────────────────────────────────────────────┘

# 排查顺序：① 这份配置走没走配置中心（还是写死本地）
#           ② 被谁在更晚阶段覆盖了（环境变量/命令行）
#           ③ 读它的 bean 在哪一刻被构造（构造后才改无效）
#           ④ 是不是 bootstrap 阶段才需要的项却写在 application
```

```
# 示意：bootstrap.yml 只放"怎么连配置中心"，不放业务配置
spring:
  application:
    name: order-service
  cloud:
    config:
      uri: http://config-server:8888   # 必须 bootstrap 阶段就可知
      profile: ${DEPLOY_ENV:dev}
```

拆解：

3.1 为什么需要配置中心

没有它，配置散落在每个实例的 `application.yml` 里，改一次要重启 N 个实例，且环境差异靠复制粘贴维护、极易出错。配置中心把配置集中到仓库、按应用/环境组织，实例启动时拉取，修改后通过刷新机制下发，避免重启。它的价值不是"能远程读配置"，而是"改配置不再等于发版"。

3.2 bootstrap 与 application 的阶段之分

这是"配了不生效"最高频的根因。`bootstrap.yml` 在框架最早阶段加载，用来告诉应用"配置中心地址、应用名"这些**引导信息**——配置中心本身的位置不能从配置中心获取，悖论，所以必须本地/命令行提前给。业务配置在 `application` 阶段拉取。若把"需要 bootstrap 阶段就读到"的项错放在 `application`，它根本没机会生效。同理，`@RefreshScope` 只刷新它标注的 bean；在构造器里一次性读走的配置，刷新不会回溯。

3.3 覆盖顺序与"被谁覆盖"

同名配置按"后加载覆盖先加载"：命令行参数 > 环境变量 > 配置中心 > 本地 `application.yml`。所以"明明配了 A，跑起来却是 B"，第一反应应是"有更高优先级的来源覆盖了我"——先查环境变量/启动参数，再查配置中心是否有同名项。

3.4 版本适配：Spring Cloud 与 Spring Boot 的硬绑定

Spring Cloud 的每个大版本只兼容特定范围的 Spring Boot。版本错配会表现为：类找不到、自动配置类不加载、`@EnableXXX` 注解无效果、方法签名不匹配。这不是你代码写错，而是依赖图本身不自洽。**不要凭直觉升级其中一个**，必须按官方版本矩阵成对地选。原书明确给出它所用的 Spring Cloud 与 Spring Boot 版本组合，就是这个原因。

今天怎么做：Nacos Config / Apollo 在"动态推送 + 灰度 + 加密"上比早期 Config Server 更强，但"引导配置必须在 bootstrap 阶段就位、配置有覆盖优先级、刷新有作用域"这三条仍然成立。版本矩阵依然硬——无论用哪套，升级时 Cloud 与 Boot 必须配套。

容易误解的地方：

1. 以为"改了配置中心，所有实例立刻生效"——需要触发刷新（`@RefreshScope` + 刷新事件），且只在标注作用域内生效。
2. 以为"`application.yml` 和 `bootstrap.yml` 随便放"——引导信息必须 bootstrap 阶段可读，放错阶段等于没配。
3. 以为"配置不生效就是我写错 key"——大概率是被更高优先级的来源（环境变量/命令行）覆盖了。
4. 以为"升级 Boot 不影响 Cloud"——二者强绑定，单独升级几乎必然出现类/自动配置不兼容。

记住这一句：配置不生效，先问"它在哪一阶段被加载、被谁覆盖、在哪个时刻被读取"——这三个问题能定位九成的"配了不生效"。

第4章　客户端负载均衡：Ribbon 与 Feign 各管一段

一句话：Ribbon 负责"把请求发给哪台机器"，Feign 负责"把方法调用变成 HTTP 请求"，Hystrix 负责"发不出去或太慢时怎么办"——三者串联，谁都替代不了谁。

为什么需要它：服务端负载均衡（Nginx 反向代理）是在入口处选机器，所有调用方共用一份决策、且只看到网关这一个上游。但在微服务内部，一个消费者要调用几十个服务、每个服务又有几十个实例，如果还靠 Nginx 做内部 LB，配置会爆炸、且无法按调用语义（如按版本、按区域）选实例。于是"选机器的决策"被下沉到**调用方进程内**——这就是客户端负载均衡。现场：你以为"加了 Ribbon 就自动均衡了"，结果所有请求都打在同一台实例——因为没配 `ServerList` 来源，Ribbon 退化为单点。现场二：你在写接口上开了重试，结果下游重复创建了两笔订单——因为重试打在了非幂等接口。现场三：服务实例列表明明变了，消费方半天不刷新——因为本地缓存 TTL 太长。这一章讲清：Ribbon 到底握着哪些抽象，以及它和 Feign、Hystrix 的边界。

核心结构：

抽象	职责
`ILoadBalancer`	维护可用服务实例集合，对外提供 `chooseServer`
`IRule`	选实例的策略（轮询/随机/重试/权重/可用优先等）
`ServerList`	实例来源，通常从注册中心（Eureka）拉取
`ServerListFilter`	对实例列表做裁剪（如按区域、zone 过滤）
`IPing`	实例存活探针，决定实例是否纳入可用集
`RestClient` / `Feign`	真正发起 HTTP 调用的客户端

关键设计：

```
一次"服务调用"在消费者进程内被切成三段

接口方法(业务代码)
    |
    ▼
[Feign]  把方法 + 参数 → HTTP 请求（URL/Header/Body）    ← 管"请求长什么样"
    |
    ▼
[Ribbon] 凭 serviceId 从本地实例缓存选一台               ← 管"发给哪台机器"
    |        ↑ 列表来自 ServerList(Eureka)，IPing 探活
    ▼
[Hystrix] 用线程池/信号量包住这次调用，超时/熔断/降级    ← 管"出事怎么办"
    |
    ▼
真实 HTTP 发送到被选中的实例
```

关键分界：**Feign 不知道"哪台机器"，Ribbon 不知道"请求内容"，Hystrix 不知道"业务语义"**。三者正交，组合成一个完整的、可防护的调用。把 Ribbon 的逻辑塞进 Feign（或反过来）只会让两者都变复杂且难测。

拆解：

4.1 为什么负载均衡要下沉到客户端

服务端 LB 的优点是调用方无感知、集中管控；缺点是它站在调用方和所有服务之间，成为流量与配置的集中点，且只能基于"到达网关的请求"做策略，看不到"我是 A 服务在调 B 的 v2 版本"这种语义。客户端 LB 把决策放进每个调用方：它能结合注册中心给的实例元数据进行细粒度选择（zone affinity、版本路由），也能针对单个调用做重试/超时。代价是：每个进程都要持有一份实例缓存并自行探活，实例信息的一致性靠"定期拉取 + 缓存"保证（因此有发现延迟，见第 2 章）。

4.2 Ribbon 的核心抽象

`ILoadBalancer` 持有 `ServerList`（实例来源）和 `IRule`（选择策略）。默认 `ServerList` 从 Eureka 拉全量实例，本地缓存并定时刷新；`IPing` 周期性探活，把不健康的剔除出可用集；`ServerListFilter` 可基于 zone 等做二次裁剪。`IRule` 的常见实现：轮询（RoundRobin）、随机（Random）、重试（Retry，对选中的实例失败后再选一个）、响应时间权重（WeightedResponseTime，给快实例更高权重）、可用实例优先（AvailabilityFiltering，跳过并发高/熔断的）。**选择哪个 Rule，本质是"你更在意公平、随机性、还是避开慢节点"的取舍。**

4.3 重试与幂等的约束

客户端 LB 天然会在"选中的实例调用失败"时换一台重试（`RetryRule` 或 Feign/Ribbon 的重试配置）。这能自动规避单实例故障，但对**写接口是危险**的：如果"创建订单"在服务端已经执行成功、只是响应在回传时丢失，重试会再创建一笔。因此：重试只应默认开启在**读接口或显式幂等**的写接口上；非幂等写接口必须关闭重试，或业务层用幂等键去重。这是客户端 LB 与幂等设计强耦合的地方。

4.4 Ribbon 与 Feign 的分工边界

Feign 负责"声明式地构造请求"——你写接口，它用动态代理把方法调用翻译成带 URL、Header、Body 的 HTTP 请求，并通过 `Client` 发出去。但"这个请求发往哪个 host:port"不是 Feign 决定的，而是它在执行时向 Ribbon 要一个 `Server`，把 serviceId 替换成具体地址。所以 Feign 是"怎么叫"，Ribbon 是"叫谁"。二者通过 `LoadBalancerClient` 衔接：Feign 拿到 Ribbon 选出的实例地址后完成最终调用。理解这条衔接线，才知道"配 Ribbon 超时不生效"往往是因为超时其实在更外层（Hystrix）先触发了。

今天怎么做：Spring Cloud 官方已将 Ribbon 标记为维护模式，推荐使用 **Spring Cloud LoadBalancer**（基于 `ReactiveLoadBalancer`，与 WebFlux/响应式更好配合），Feign 仍通过 `spring-cloud-openfeign` 使用。但"LB 决策在调用方、实例来自注册中心缓存、重试必须配合幂等"这套判断标准不变。Kubernetes 场景下，常直接用 Service DNS + kube-proxy 或 Istio 做 LB，但"重试与幂等"的约束依然是你的责任。

容易误解的地方：

1. 以为"Ribbon 会自动均衡，所以随便调"——若没接 `ServerList`（如没连注册中心），它只剩一个写死的 server，谈不上均衡。
2. 以为"客户端重试是免费的"——在写接口上重试会导致重复下单/重复扣款，必须显式关掉或加幂等键。
3. 以为"Ribbon 超时配了就生效"——当 Feign+Hystrix 叠加时，最先触发的那一层超时决定结果，Ribbon 的超时可能被 Hystrix 先截断（见第 5 章）。
4. 以为"负载均衡只关心算法"——算法只是 `IRule`，实例来源、探活、过滤、缓存刷新频率共同决定"你到底在哪些实例里选"。

记住这一句：Feign 决定请求长什么样，Ribbon 决定发给哪台机器，Hystrix 决定出事怎么办——把这三段拆开看，调用链上的"不生效"和"雪崩"才各有归处。

第5章　Feign 动态代理：一个接口怎么变成一次 HTTP 调用

一句话：一个只写了注解的接口，被 JDK 动态代理包成 `InvocationHandler`，方法调用时现拼出 HTTP 请求并交由 `Client` 发出——三套超时（Feign/Ribbon/Hystrix）里谁小谁先说话。

为什么需要它：你只写了 `@FeignClient("order-service") interface OrderApi { @GetMapping("/{id}") Order get(@PathVariable Long id); }`，没有实现类，却能在注入后直接 `orderApi.get(1L)` 拿到远程结果。魔法全在"动态代理"。现场：你给 Feign 配了 3 秒 `readTimeout`，压测还是 1 秒就熔断——因为真正兜底超时的是 Hystrix。现场二：你想给所有请求统一加鉴权头，不知道该在哪一环插手——答案是 `RequestInterceptor`。现场三：你怀疑请求 URL 拼错，却找不到"拼接逻辑在哪"——它散落在 `Contract`、`RequestTemplate`、`Encoder` 之间。这一章把这条流水线从"接口"到"字节"逐段拆开。

核心结构：

组件	职责
`Contract`	解析接口上的注解，生成 `MethodMetadata`（URL 模板、HTTP 方法、参数索引）
`RequestTemplate`	一次调用的可变的请求模板（URL、Query、Header、Body）
`Encoder` / `Decoder`	把对象编成请求体、把响应体解回对象
`RequestInterceptor`	在发出前统一加工请求（加 Header、加 Token）
`InvocationHandler`（ReflectiveFeign）	JDK 动态代理的调用入口，把方法调用路由到 `MethodHandler`
`MethodHandler`（SynchronousMethodHandler）	每个方法一个，负责拼模板、拦截、调用 `Client`
`Client`	真正发 HTTP 的实现（默认 `HttpURLConnection`，可换 OkHttp/Apache）
`Target`	把 serviceId 解析成基础 URL（结合负载均衡）

关键设计：

```
// 示意：Feign 代理的骨架（非原书逐字，仅为说明层级）
public class ReflectiveFeign implements InvocationHandlerFactory {
  // 每个接口方法 → 一个 SynchronousMethodHandler
  public Object invoke(Object proxy, Method method, Object[] args) {
    MethodHandler handler = dispatch.get(method);   // 按方法路由
    return handler.invoke(args);                    // 交给对应处理器
  }
}

// SynchronousMethodHandler.invoke 的核心步骤
RequestTemplate template = targetRequest(args);      // 1. 用 Contract 生成的元数据拼模板
template = applyInterceptors(template);              // 2. RequestInterceptor 统一加头/改写
Client client = loadBalancedClient();                // 3. 带 Ribbon 的 Client：先选实例
Response resp = client.execute(template.request());  // 4. 真正发 HTTP（serviceId→具体地址）
return decode(resp);                                 // 5. Decoder 把响应解回对象 / 走 errorDecoder
```

注意第 3 步：当 Feign 与 Ribbon 配合时，`Client` 是"带负载均衡的"——它内部调用 Ribbon 选出实例，把 `http://order-service/{id}` 换成 `http://10.0.1.7:8080/{id}` 再发。

拆解：

5.1 为什么必须是接口 + 动态代理

JDK 动态代理要求目标必须是接口，它在运行时生成一个"实现该接口"的代理类，把所有方法调用收口到一个 `InvocationHandler`。Feign 借此做到"你只声明、我不写实现"：编译期不存在任何实现类，运行时才由代理把方法调用翻译成 HTTP。这也意味着——你不能在接口方法里写逻辑，所有行为都来自代理链的组装（Contract 解析、Interceptor 加工、Client 发送）。好处是零侵入、可组合；代价是调试时要理解这条"看不见的链"。

5.2 代理实例的创建流程

`@EnableFeignClients` 扫描所有 `@FeignClient` 接口，每个接口由一个 `FeignClientFactoryBean` 负责。它拿到配置（日志级别、拦截器、编解码器、超时 Options 等），构建 `Feign.Builder`，最终 `target()` 生成代理对象放进容器。`Builder` 是可插拔的：换 `Client`（OkHttp）、加 `Logger`、`RequestInterceptor`、自定义 `Contract` 都在这一步完成。理解"创建"和"执行"是两个阶段，就能解释为什么有些配置（如超时）要在创建时就定好、运行时改不动。

5.3 一次调用的执行流程

方法被调用 → `InvocationHandler` 按方法找到 `MethodHandler` → `SynchronousMethodHandler` 用 `MethodMetadata` 把参数填进 `RequestTemplate`（Query 参数、Path 变量、Body）→ 依次过所有

`RequestInterceptor`（统一加鉴权头、traceId 等）→ `Client.execute` 发出 → `Decoder` 把 2xx 响应解回对象；非 2xx 走 `ErrorDecoder`（默认对 4xx/5xx 抛 `FeignException`，5xx 才可能触发重试/熔断）。每一步都是独立可替换的组件，这正是 Feign "声明式但高度可定制"的来源。

5.4 超时三件套：谁小谁说了算

这是"配了不生效"最高频的坑。叠加 Feign + Ribbon + Hystrix 时存在三套超时：

- **Feign 的 `Options`（connect/read）**：仅当 Ribbon 未介入、使用默认 `Client` 时直接生效；一旦走 Ribbon，`Client` 实际用的是 Ribbon 的 `ConnectTimeout` / `ReadTimeout`。
- **Ribbon 的 `ConnectTimeout` / `ReadTimeout`**（默认 1000/1000ms 量级）：真正控制底层 socket 超时的，常常是它。
- **Hystrix 的 `execution.timeout`**（默认 1000ms）：它在最外层包住整个调用，超时直接走 fallback。

结论：**Hystrix 超时 < Ribbon 超时 < Feign 超时** 时才不浪费；若 Hystrix 超时最小，调用常常没等到底层超时就被熔断了——你以为"3 秒超时"，其实 1 秒就 fallback。同理 Ribbon 的重试次数 × 单次超时 也得小于 Hystrix 超时，否则重试还没跑完就被熔断截断。

今天怎么做：OpenFeign + Spring Cloud LoadBalancer 仍是主流组合，超时层级（Feign Options / LoadBalancer 配置 / Resilience4j `TimeLimiter`）依然存在且必须按大小关系排布；`RequestInterceptor` 仍是统一加认证/链路追踪头的标准位置。响应式场景下 Feign 被 WebClient/声明式 HTTP 客户端替代，但"声明 → 拼请求 → 拦截 → 发送 → 解码"的流水线结构没变。

容易误解的地方：

1. 以为"给 Feign 配了 readTimeout 就一定生效"——走 Ribbon 时实际是 Ribbon 的 socket 超时在起作用。
2. 以为"超时配大一点更安全"——在 Hystrix 包裹下，Hystrix 超时最小，配大 Feign 超时只是让线程被占用更久，不解决熔断。
3. 以为"动态代理是 Feign 自己实现的黑魔法"——它就是标准 JDK `Proxy` + `InvocationHandler`，接口是唯一前提。
4. 以为"加鉴权头要改每个方法"——用 `RequestInterceptor` 在调用前统一加，才是正解。

记住这一句：Feign 把"方法调用"翻译成"HTTP 请求"靠的是 JDK 动态代理 + 一条可替换组件的流水线；三套超时里最小的那一个，才是你真正体验到的超时。

第6章　RPC 保护：舱壁、熔断器与滑动窗口

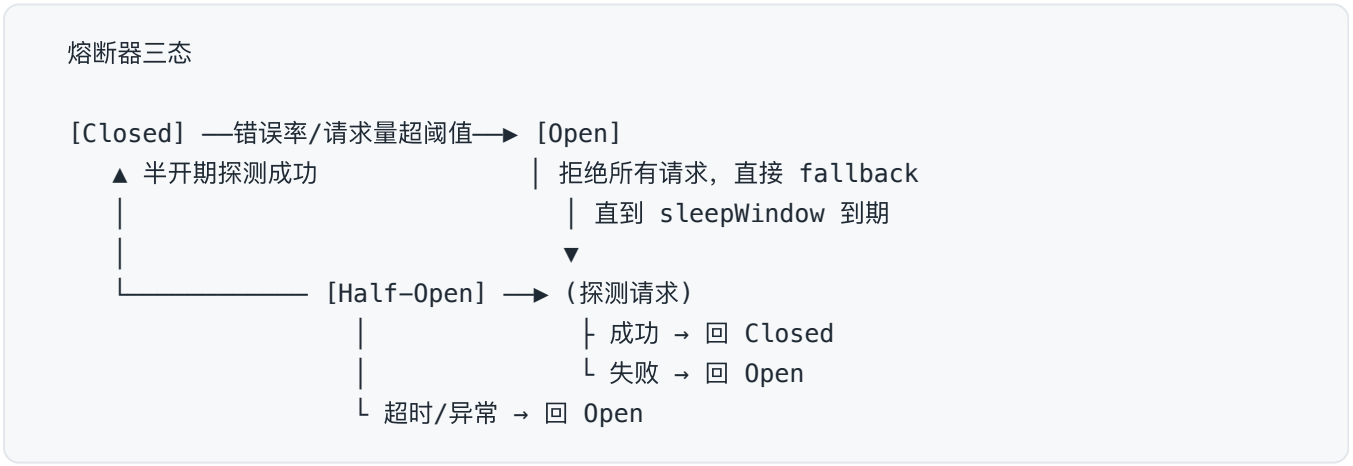
一句话：RPC 保护不是"不让下游出错"，而是"不让下游的慢把上游的资源耗尽"——舱壁隔离资源、熔断切断失败调用、滑动窗口提供统计依据。

为什么需要它：没有保护时，一个下游服务变慢，上游每个请求都卡在等它响应，线程被持续占用；并发一高，上游线程池被打满，连本来正常的其他接口也全挂——这就是**雪崩**。现场：订单服务依赖库存服务，库存服务一次慢查询，订单服务的 Tomcat 线程全卡在等库存，结果下不了单也查不了订单。现场二：你把超时设很大"以防误杀"，结果雪崩更彻底——线程更久才释放。现场三：熔断后你写了个 fallback 又去调另一个远程服务，结果把熔断的下游压力转嫁成了新下游的雪崩。这一章讲清 Hystrix 的三板斧：隔离、熔断、降级，以及它们各自挡住的是哪一类失败。

核心结构：

手段	挡住的问题	代价
舱壁隔离（线程池/信号量）	一个慢依赖耗尽上游全部线程	额外线程/队列开销、上下文切换
熔断器（Closed/Open/Half-Open）	对已经失败的依赖持续发送请求	半开期的探测流量、误开后的拒绝
降级 fallback	主逻辑不可用时的兜底返回	兜底逻辑若再调远程会制造新雪崩
超时	单请求无限等待	过早失败的误判

关键设计：



滑动窗口：把时间切成若干小桶（bucket），每个桶独立计数成功/失败/超时；熔断器看的是"最近 N 个桶的滚动统计"，而不是从启动到现在的总账——这能及时反映"刚刚开始变差"的依赖，又不会被很久以前的正常历史稀释。

拆解：

6.1 雪崩的传导路径

上游线程被下游阻塞 → 上游线程池耗尽 → 上游对所有请求（含健康依赖）都拒绝 → 上游自身被拖死。根因不是"下游慢"本身，而是"上游用自己有限的线程，去无限等待一个无限的慢"。RPC 保护的出发点，就是**给每个依赖划一道资源边界和失败边界**，让一个依赖的恶化被关在它自己的笼子里，不溢出到上游全局。

6.2 舱壁模式：隔离而非共享

舱壁（bulkhead）取自船舱隔水：一个舱进水不淹全船。两种实现：

- **线程池隔离**：每个依赖（或命令组）独占一个线程池。依赖慢只会撑满自己的池，上游主线程（如 Tomcat 线程）立刻拿到 fallback，不被阻塞。优点隔离彻底；代价是线程多、上下文切换、且不适合高并发短调用（线程创建开销）。
- **信号量隔离**：用计数器限制"同时有多少个调用能进入该依赖"，不另起线程。开销小、无切换，但调用仍运行在调用方线程上，若依赖阻塞，信号量也只是限制并发数、并不释放调用方线程——所以信号量隔离不能隔离"阻塞"，只隔离"并发量"。**选哪个取决于你要隔离的是'阻塞'还是'并发'。**

6.3 熔断器：切断已经失败的调用

Closed 状态下请求正常放行并统计；当"时间窗内请求量达到阈值且错误率超过阈值"，切到 Open，此后请求直接走 fallback 不再打下游，给下游喘息。Open 持续一个休眠窗口（sleepWindow）后进入 Half-Open，放一个探测请求：成功则回到 Closed，失败则继续 Open。关键参数：**请求量阈值**（避免"只失败 1 次就熔断"的误判）、**错误率阈值**（区分偶发与持续失败）、**休眠窗口**（决定多久试探一次）。它挡的是"对已知失败的依赖反复无效请求"——把"必然失败"变成"立刻失败+兜底"。

6.4 滑动窗口与降级的反模式

熔断器依赖"最近一段时间的失败比例"，固定窗口（如每分钟重置）会有边界突刺问题（59 秒失败 99 次、第 60 秒重置，第 61 秒又放行）——滑动窗口用滚动桶解决，让统计连续反映近期状态。降级 `fallback` 是兜底返回（如返回缓存、默认值、空列表），**但 fallback 里再去调另一个远程服务是反模式**：你把熔断保护掉的下游压力，原封不动转嫁成了新下游的雪崩。fallback 应当尽量本地化（缓存/默认值/快速失败），而非链式远程调用。

今天怎么做：Hystrix 已进入维护模式，主流替换为 **Resilience4j**（基于函数式、无额外线程池开销，用 `TimeLimiter` + `Bulkhead` + `CircuitBreaker`）或 **Sentinel**（流量控制 + 熔断 + 降级一体，支持基于 QPS/并发数的规则）。但"隔离资源、熔断切断失败调用、降级走本地兜底、统计用滑动窗口"这四件事及其取舍完全不变。

容易误解的地方：

1. 以为"熔断=超时"——超时是单请求边界，熔断是对"依赖整体健康"的判断，两者互补不替代。
2. 以为"把超时设大更稳妥"——越大，线程被占用越久，雪崩越彻底；正确做法是短超时 + 熔断 + 快速失败。
3. 以为"信号量隔离和线程池隔离差不多"——信号量不隔离阻塞，只在乎并发数；要隔离'卡住'，必须线程池。
4. 以为"fallback 里再调一个服务做补偿很自然"——这是把雪崩平移，而非消除。

记住这一句：RPC 保护的三件事各管一段——舱壁把慢依赖关进自己的资源笼子，熔断切断对失败依赖的无效请求，降级用本地兜底替代远程失败；它们合起来，才挡得住雪崩的传导。

第7章　响应式与背压：RxJava 在调用链里的位置

一句话：RxJava 不负责更快，它把异步执行、滑动窗口统计和速率协商统一到观察者模式上。

为什么需要它：秒杀链路已经用 Hystrix 做了线程池隔离（bulkhead），压测时命令成批超时，可熔断器（circuit breaker）迟迟不打开。翻到指标采集才发现，"最近一个时间窗内的错误率"并不是每来一次请求就算一次，而是一条按时间不断滚动的事件流——窗口没滚过去，错误率就不会更新，熔断自然滞后。再把一个无限循环的事件源接到每 50 毫秒才消费一条的下游上，几秒钟内存就被异步缓冲池吃满，抛出 MissingBackpressureException，而系统既没有拒绝策略也没有降速手段。这两处都指向同一个东西：Spring Cloud Netflix 体系里的 Hystrix 与 Ribbon，其异步/响应式执行、滑动窗口（sliding window）统计、以及高并发下的异步编排，全部建立在观察者模式（observer pattern）之上。不懂这条事件流，就只能在配置项层面猜"为什么配了不生效"。这一章把观察者模式、操作符、调度器（Scheduler）与背压（backpressure）拆成可迁移的判断：谁在推事件、事件在哪条线程被消费、消费跟不上时由谁让步。

核心结构：

概念	作用
Observable（被观察者/主题）	产生事件序列，内部持有弹射器与缓存队列，对应 GoF 的 Subject 角色
Subscriber / Observer	通过 subscribe() 建立订阅，以 onNext / onCompleted / onError 被动接收
Emitter 弹射器	onNext 推数据；onCompleted 与 onError 是终止信号，互斥且必须是序列最后一条
Subscription 订阅关系	subscribe() 返回它，可 unsubscribe() 取消订阅，断开生产与消费
操作符 operator	对事件流做变换与组合，返回新的 Observable，而不是改写原流
Scheduler 调度器	决定事件在哪个线程产生、在哪个线程被消费
Emitter.BackpressureMode	定义消费跟不上时的速率协商：报错、缓冲、丢弃、保留最新
滑动窗口 sliding window	把连续事件按数量或时间切成桶，Hystrix 的健康统计即基于此

关键设计：

```
  [生产者 Observable]            [操作符链]              [消费者 Subscriber]
        |                             |                          ^
        | onNext(v)                   | map/filter/window        | onNext(v')
        | 推：生产端主动               | 变换：返回新流            | 被动回调
        v                             v                          |
  subscribeOn(Schedulers.io()) -> 产生线程：IO 线程池
                                  observeOn(computation()) -> 消费线程：计算线程池
                                   （其后最近一次生效，可多次切换）

  onCompleted / onError：终止信号，互斥，必须是序列最后一条
  失衡：生产快于消费 -> 事件堆在 observeOn 的缓冲队列 -> 背压在此产生
```

```
// 示意：调度归属与订阅，非原书代码
source                                    // 创建型操作符：事件在此产生
  .subscribeOn(Schedulers.io())           // 只决定"产生"线程；多次写只认最近一次
  .map(v -> v * v)                        // 转换型：逐项变换，一对一
  .window(3)                              // 把流切成桶，统计窗口由此而来
  .observeOn(Schedulers.computation()) // 只决定其后"消费"线程；可多次切换
  .subscribe(new Subscriber<String>() {
      public void onNext(String s) { /* 推过来的，不是我去拉 */ }
      public void onCompleted() { }
      public void onError(Throwable e) { }
  });
```

注释里的"为什么"：subscribeOn 与 observeOn 作用方向相反，是因为订阅关系自下而上建立、事件自上而下流动；把两者混为一谈，就会出现"切了线程却依旧卡在 IO 上"。

拆解：

推与拉：观察者模式凭什么撑起 Hystrix

经典观察者模式有四个角色：Subject（抽象主题）维护 Observer 集合并提供增删与通知接口，ConcreteSubject 持有状态并在变化时通知，Observer 定义更新接口，ConcreteObserver 实现真正的更新。RxJava 把 Observable 与 Subscriber 分别对应到 Subject 与 Observer 两个角色，用 subscribe() 建立订阅。关键差别有两点。第一，经典主题一次通知只送一个消息，而 RxJava 的 Emitter 用 onNext() 连续弹射一串消息组成"消息流"，主题内部还有队列缓存，这已经借了生产者/消费者模式的思想；onCompleted() 表示流正常结束，onError() 表示异常终止，二者互斥且必须是最后一条。第二，不完整回调（Action0 / Action1 / ActionSubscriber）把三个回调拆成函数式接口，最终仍被包装成一个 Subscriber——这是语法糖，不是另一套模型。订阅关系不是永久的：subscribe() 返回一个 Subscription，调用 unsubscribe() 就断开这条订阅，生产端不再向这个消费者推送，这是推模型里唯一由消费端主动行使的权力，也是背压得以介入的前提。

为什么一本讲 Spring Cloud 的书要讲它？因为 Hystrix 用命令模式结合 RxJava 与滑动窗口技术：HystrixCommand / HystrixObservableCommand 的执行本身是异步事件流，超时、回退、成功、失败都是一个个事件；滑动窗口把这些事件按桶聚合，HealthCounts 里的总请求数与错误比例才随之滚动。熔断判断读的

是这条流，不是实时计数器。看懂订阅关系与取消订阅，才能解释"命令已经超时，为什么窗口还没把这次失败计进去"——失败事件还在缓冲队列里没被窗口消费，判断依据自然滞后。

维度	迭代器 Iterator（拉）	观察者 Observable（推）
数据流向	消费者调用 next() 主动取	生产者调用 onNext() 主动送
同步性	天然同步，取一步算一步	可同步可异步，常跨线程
节奏控制方	消费者：不取就不生产	生产者：推多快由它定
结束信号	hasNext() 为假	onCompleted / onError
组合方式	嵌套循环、外部游标	操作符链式变换，返回新流
阻塞点	取不到就阻塞调用线程	消费慢则事件积压在缓冲队列
需要的保护	无	背压：消费者跟不上时的速率协商
典型代表	for 循环遍历集合	Hystrix 命令流、消息流、指标流

推模型的代价就写在最后两行：节奏不在消费者手里，所以必须额外发明背压；迭代器天然不会积压，因为"不取"本身就是减速。这也解释了为什么响应式代码更容易在高并发下暴露内存问题——生产端默认不会等。

操作符按职责分类：变换与组合，不是 API 清单

原书按创建型、过滤型、转换型、聚合型分节讲操作符，逐个罗列 API 反而淹没共性。可迁移的判断只有一句：操作符本质是对事件流的变换与组合，每个操作符接收一个 Observable、返回一个新的 Observable，原流不被改写。按职责记四类就够。创建型（create、just、from、range、interval、defer、empty、error、never）负责"从无到有"或"何时才有"——defer 尤其值得记：它在订阅发生时才创建主题，每个订阅者拿到独立的序列，所以能读到订阅时刻的最新值，而 just 在创建时就把值定死了；原书那个 AtomicInteger 从 100 改到 200 的例子，just 仍弹 100、defer 弹 200，差别只在"何时读值"。过滤型（filter、distinct）按条件或去重筛掉事件，流的形状不变、元素变少。转换型（map、flatMap、scan）改变元素或结构：map 是一对一，flatMap 把一个元素摊成一个新流再合并，是"一对多且可换类型"，scan 带累积状态、把每一步中间结果都弹出来。聚合型（count、reduce）把整条流收敛成单个值，且必须等源流终止才弹出——reduce 与 scan 的差别只在"弹每一步"还是"只弹最终值"。另有 take / skip 按索引截取、window 按数量或时间分桶，window(count, skip) 还能做重叠的滑动窗口，这正是 Hystrix 指标窗口的同构写法。

分类之所以比罗列重要，是因为它给出组合规则与排障顺序：创建定源头、过滤定取舍、转换定形态、聚合定出口，任何一条调用链都能按这四段读，而不需要记住几十个方法名。看到数据少了先查过滤段，看到类型对不上先查转换段，看到"统计一直不出数"先查是不是用了必须等流终止的聚合操作符。还有一层判断：flatMap 因为内部创建了新 Observable，原书甚至把它归到创建型——这说明分类是按"这一刻在干什么"而不是按方法名字面归属，同一操作符在不同视角下可以落进不同类。

Scheduler 与背压：谁定节奏、跟不上怎么办

Scheduler 回答的是"在哪个线程上产生/消费事件"，与"快不快"无关。工厂方法给出的语义要先分清：io() 是可复用、可无上限增长的缓存线程池，用于阻塞式 I/O；computation() 是固定为 CPU 核数的线程池，只该跑计算，不该拿来解析 JSON 或压图片这类会占着线程等 I/O 的活；newThread() 每次新建线程、没有复用，创建成本高于复用；trampoline() 在当前线程排队、等当前任务跑完再执行；single() 是单例线程、任务按 FIFO 排队。两个切换点的语义差别是本章最容易错的地方：subscribeOn() 改变"弹射/产生"的线程，由创建操作符之后最近的一次 subscribeOn 决定；observeOn() 改变其后"下游消费"的线程，可以多次设置，每次只对到下一次 observeOn 之间的操作生效。原书那句"subscribeOn 主要改变弹射的线程、observeOn 主要改变订阅的线程"，说的就是作用方向相反，不是两个都切消费端。

为什么线程池隔离天然需要这层调度？bulkhead（舱壁/线程池隔离）的本质是把一次远程调用从请求线程挪到独立的命令线程池去执行，配置 ribbonIsolationStrategy: THREAD 正是这个选择：命令在独立线程池里跑，超时才不会吃掉容器线程，Hystrix 也才能拿到一个干净的"超时"事件去喂滑动窗口。挪线程这件事在 RxJava 里就是一次 subscribeOn；没有调度器，命令的执行与回退就只能挤在容器线程上，隔离无从谈起，信号量隔离则省掉这次切换、代价是无法对慢调用做硬超时。两种隔离的取舍，本质是"要不要为可控的超时付出线程切换与上下文传递的成本"。

背压（backpressure）是消费者跟不上生产者时的速率协商：上下游处于不同线程时，来不及处理的事件既不会丢也回收不掉，堆在异步缓冲池里直至内存溢出。原书把模式枚举成 Emitter.BackpressureMode，三种策略各有代价。无界缓冲（BUFFER，初始 128 后无限增长）不丢数据，代价是内存不可控，持续积压最终 OOM；丢弃（DROP 保旧丢新、LATEST 保新丢旧，都用固定 128 缓冲）内存可控，代价是数据不完整，LATEST 适合"只要最新状态"的行情、倒计时类场景；直接报错（NONE / ERROR 抛 MissingBackpressureException）把问题暴露成故障，代价是可用性。判断标准是先问三件事：数据能不能丢、丢谁、能不能让上游慢下来。

它和限流（rate limiting）不是一回事：背压是链路内部消费者向生产者的反馈，作用范围是一条事件流的两端，目的是"别把中间压垮"；限流是在系统入口按容量直接拒绝超额请求（另有专章讲令牌桶、漏桶与分布式计数器），目的是"别让流量进来"。一个保护链路，一个保护系统容量；把背压当限流用，入口照样会被打穿，因为链路内部再怎么协商，都拦不住已经进门的请求。

今天怎么做：Project Reactor、WebFlux 与"响应式不等于更快"

今天怎么做：Netflix 这套组件已进入维护模式，工程上的映射是 Spring Cloud LoadBalancer 取代 Ribbon、Resilience4j / Sentinel 取代 Hystrix、Spring Cloud Gateway 取代 Zuul，而响应式基础从 RxJava 换成了 Project Reactor 与 WebFlux。变的是实现：Reactor 把背压做成消费者显式声明需求量、上游按需推送，比 RxJava 1.x 用枚举选 BackpressureMode 更成体系；不变的是职责边界——观察者式的推、操作符对流变换、调度器定线程、背压定速率，四件事一个都没少。所以读原书这章的价值不在记住 create / flatMap 的方法签名，而在能一眼看穿新框架里哪一层在做同样的事。

必须说清两个判断。响应式不等于更快：它省的是"阻塞等待的线程"，不是单次调用的计算量；当下游是慢数据库或强 CPU 运算时，吞吐上限仍由下游决定，事件并不会凭空少消费一次，背压只是把"堆死"换成"协商"。什么时候不该用：业务逻辑以顺序、阻塞、事务为主，团队对异步调试与栈追踪不熟，或者链路里根本没有需要编排的并发 I/O 时，引入响应式只会换来更难排查的超时和更高的认知成本。判断标准始终是"有没有需要被编排的异步边界"，而不是"框架流不流行"。

容易误解的地方：

- 很多人以为用了响应式请求就更快，其实它只改变线程编排与阻塞方式，因为吞吐仍由下游和线程池上限决定。
- 很多人以为 subscribeOn 和 observeOn 都是切"消费"线程，其实前者定产生端且只认最近一次、后者定其后消费端且可多次，因为订阅自下而上建立、事件自上而下流动。
- 很多人以为背压就是丢弃多余数据，其实丢弃只是三种代价之一，因为要先决定数据放哪、能否丢、能否让上游减速。
- 很多人以为背压等于限流，其实前者是链路内部的速率反馈、后者是入口处的直接拒绝，因为一个保护链路一个保护容量。

记住这一句：背压谈的是链路里谁等谁，响应式谈的是事件怎么被推着走——快慢是结果，节奏归属才是设计。

第8章　网关：Zuul 过滤器链与路由边界

一句话：网关把鉴权、路由、限流、日志收成一条过滤器链，代价是它自己成了必须被冗余的单点。

为什么需要它：第 12 个 Provider 上线时，运维照例又配了一遍 token 校验、跨域白名单、访问日志和限流阈值；四份逻辑散在 12 个仓库里，改一次校验规则要发 12 次版。更要命的是客户端自己维护实例列表并做负载均衡，一个实例已经下线，请求还在往它发，报错散落在每个调用方。把这些横切关注点（cross-cutting concern）收进一个入口后，问题并没有消失，只是换了形态：Nginx 把 80 端口的流量全部 proxy_pass 到唯一的 Zuul upstream，一旦这个进程挂了，整站所有服务同时不可达；而某个 pre 过滤器里顺手写了订单状态判断之后，每条业务线改需求都要重启网关。这一章要回答的是：外部网关与内部网关各自该管哪一段，四类过滤器按什么契约串成一条链，路由规则怎么和服务发现拼起来，以及为什么在网关里写业务逻辑、把网关当单点，是两种必须提前挡住的设计。

核心结构：

概念	作用
外部网关 Nginx / OpenResty	扛连接、TLS、静态资源、keepalive，反向代理（reverse proxy）到 Zuul
内部网关 Zuul（@EnableZuulProxy）	服务级路由 + 业务横切，自身也是注册到 Eureka 的微服务
ZuulFilter	自定义过滤器抽象基类，四个契约方法
filterType / filterOrder	阶段类型；同类型内按值升序，越小越先执行
shouldFilter / run	是否生效的开关；真正承载拦截、改请求、改响应的逻辑
RequestContext	每请求唯一，过滤器之间不直接通信、只共享这个上下文
RibbonRoutingFilter	route 阶段结合 Ribbon 选实例、Hystrix 做隔离并发起调用
serviceId → 实例列表	路由规则结合服务发现（service discovery）做动态转发
sensitiveHeaders / strip-prefix	屏蔽敏感请求头；决定是否剥掉路由路径前缀
error 过滤器	任一阶段抛异常后统一接管，可直接生成最终响应

关键设计：

```
客户端 -> Nginx(外部网关) -> Zuul(内部网关) -> Provider 实例
                                  |
pre 阶段   [同类按 filterOrder 升序]  shouldFilter? -> run()
   |  鉴权 / 黑名单 / 灰度标 / 追加 USER-ID 头
   |  ctx.setSendZuulResponse(false) -> 后续 route 过滤器全部跳过
   v
route 阶段  RibbonRoutingFilter：serviceId -> 实例列表 -> 选定并调用
   |          (Ribbon 负载均衡 + Hystrix 线程池隔离 / 超时)
   v
post 阶段  加响应头 / 收集指标与统计 / 把结果回写客户端
   |
任意阶段抛异常 ----> error 阶段（统一处理，可返回最终结果）

每请求一个 RequestContext：过滤器间不互相调用，只读写这个共享上下文
```

```yaml
# 示意：路由规则，非原书完整配置
zuul:
  ribbonIsolationStrategy: THREAD           # 命令调到独立线程池，隔离由此而来
  host:
    connect-timeout-millis: 600000         # 建连超时；调大不等于下游更快
    socket-timeout-millis: 600000          # 读响应超时；超时边界在 route 阶段
  routes:
    user-provider:
      path: /user-provider/**              # 匹配前缀
      serviceId: user-provider            # 交给服务发现解析成实例列表
      strip-prefix: false                 # 保留前缀，否则上游带 context-path 时 404
      sensitiveHeaders: Cookie,Set-Cookie,Authorization  # 外发时剥掉会话凭证
```

注释里的"为什么"：serviceId 与 url 是两条不同的路，只有前者会走 Eureka 的实例列表；strip-prefix 默认 true，若上游服务自己配了 context-path 又剥掉前缀，直接 404。

拆解：

内外分界：外部扛连接，内部做服务级路由

网关解决的是横切重复：每个服务都要做鉴权、路由、限流、日志，与其在 N 个仓库各写一遍，不如收敛到一个入口，顺带把很多 Provider 的不同端口统一成网关的一个端口、把跨域处理收在一处。但"一个入口"其实是两层。外部网关是 Nginx / OpenResty，它面对的是公网连接数、TLS 握手、keepalive、静态资源与慢客户端，配套配置里 upstream zuul 挂着长连接、proxy_read_timeout 控制等待上游的时间，它擅长用 epoll 与少量 worker 扛住海量连接，却不该知道"user-provider 此刻有几个实例"。内部网关是 Zuul，它自身也是注册到 Eureka 的微服务，靠 @EnableZuulProxy 开启，负责把 /user-provider/** 这样的路径映射到具体服务、并在转发前做业务横切。原书把两者分别叫外部网关与内部网关，分工一句话：一个扛连接与静态流量，一个做服务级路由与业务横切。

为什么不能合并成一层？Nginx 跑在 Lua 协程里，适合连接层与通用限流，但每加一条业务校验都要改 OpenResty 脚本并发版；Zuul 跑在 JVM 里，能用 Java / Groovy 动态读取、编译、运行过滤器，复用 Spring

生态，却不该把宝贵线程耗在慢连接与静态文件上。两层各守住一段，才有"改鉴权只发网关、扩连接只动 Nginx"的独立演进；把它们压成一层，任何一侧的容量问题都会直接传导到另一侧。

代价也必须写清楚：Zuul 成了唯一入口，就是单点（single point of failure）。可用性的答案是它自己不留状态——鉴权结果放请求头、会话放外部 Redis，于是 Zuul 可以横向起多实例；Nginx 用 upstream 做负载均衡与健康检查，挂掉一个实例只影响在途请求；再叠一层 Hystrix 或限流让网关在下游雪崩时先自保，而不是被拖死。判断标准是"任意一台网关进程消失，是否只是一次重试的事"；如果答案是"要人工改配置"，说明状态还粘在网关里，分界就没做干净。

四类过滤器、执行顺序与错误归属

Zuul 的大部分功能本身就是过滤器实现的，连"转发"也是 route 类型过滤器。四类标准类型定义在 FilterConstants 里：PRE_TYPE、ROUTE_TYPE、POST_TYPE、ERROR_TYPE，另外还允许自定义类型（例如直接在网关生成响应、不转发上游的 echo 类）。请求先走 pre（路由前，做校验、加头、记调试信息），再走 route（构造并发送请求到上游），拿到结果后走 post（加工响应、收集指标、回写客户端）；任意阶段抛异常，就改走 error 阶段，error 过滤器同样能把最终结果返回给客户端。自定义过滤器继承 ZuulFilter，实现四个契约：filterType() 返回阶段，filterOrder() 返回顺序（值越小优先级越高），shouldFilter() 返回是否生效，run() 写逻辑。把"是否生效"独立成一个方法很关键：黑名单过滤器可以只在 URI 命中某前缀时才跑，避免每条请求都付一遍查库成本；也可以先判断 ctx.sendZuulResponse()，若前面的过滤器已经终止了请求，本过滤器直接返回 false 不再空转。

过滤器之间不直接通信，靠 RequestContext.getCurrentContext() 拿到每请求唯一的上下文。pre 里判定不通过时调用 ctx.setSendZuulResponse(false)，这个标志位会被几乎所有内置过滤器检查——例如 RibbonRoutingFilter 的 shouldFilter() 里就含 ctx.sendZuulResponse() 判断，为 false 就不再发起调用。于是一次拦截不是抛异常中断，而是"后续 route 集体让路"，pre 自己写好响应体即可。这也是 RequestContext 的职责：传状态、传开关、传请求头（addZuulRequestHeader）、传 serviceId 与 routeHost，而不是传对象引用。正因为状态集中在一处，过滤器才能动态加载、彼此零耦合。

过滤器类型	时机	典型职责	失败后果
pre	路由之前	鉴权、黑名单、参数校验、灰度标、追加 USER-ID 头	误放行则越权；误拦截则整条链被 setSendZuulResponse(false) 掐断
route	转发之时	serviceId 解析、选实例、发起调用、超时控制	超时或选错实例直接抛异常，转入 error 阶段
post	上游返回之后	加响应头、收集指标、改写响应、回写客户端	此处再抛异常会进 error，且响应可能已开始写出，处理要幂等
error	任一阶段异常后	统一异常映射、兜底响应、告警	自身再异常则无兜底，客户端拿到容器级错误

这张表真正的用途是排障：看到"鉴权通过却 404"先查 route 与 strip-prefix，看到"响应头没加上"先查 post 是否被 error 顶掉，看到"重复回写"先查 post 与 error 有没有都写了响应。把四类按"时机 + 失败后果"记，比按方法名记更能定位问题落在哪一段。

路由与服务发现、超时重试边界，以及"网关写业务逻辑"的反模式

路由规则有两条路。url 直连把 /blog/** 之类前缀映射到固定上游地址，简单、但不感知实例上下线；serviceId 方式把路径映射到服务名，由 Eureka Client 拉取实例列表、Ribbon 选一个实例转发，这才是动态路由，也是"服务已经下线、消费端还在往它发请求"的解药——实例列表由注册中心维护，网关不必硬编码 IP。配置里 serviceId 与 url 互斥，前者必须配合 @EnableDiscoveryClient（@EnableZuulProxy 已默认导入）。前缀处理是另一个高频坑：strip-prefix 默认 true，会剥掉 /demo-provider 这类前缀；若上游服务自己配了 context-path，剥掉就 404，必须显式设 false。sensitiveHeaders 则相反——它列的是"要被剥掉"的头，默认 Cookie、Set-Cookie、Authorization 不外泄；内部转发想保留原始头，要把该配置置空，而发往外部站点的规则要单独把凭据类头挡住，局部配置覆盖全局配置。这个细节决定了"配了 header 透传却不生效"的原因往往不在代码，而在网关替你把头清掉了。

超时与重试的边界在 route 阶段：connect-timeout-millis 管建连、socket-timeout-millis 管等响应，两者都只约束"网关等下游"这一段，不约束 Nginx 等网关、也不约束客户端等整条链——把它们调大只是让网关多等一会儿，不会让下游更快，反而会占住更多线程。重试必须建立在一个前提上：被重试的操作幂等，否则一次超时重放会变成两次下单；而且重试只能在 route 内做，post 已经拿到响应后再重试等于重复回写。把 ribbonIsolationStrategy 设为 THREAD，是让命令在独立线程池里执行，超时才不会吃掉容器线程；这与信号量隔离的取舍在于是否值得为硬超时付出线程切换与上下文传递的代价。

为什么"网关里写业务逻辑"是反模式？因为网关是所有服务共用的基础设施，一旦把订单状态判断、库存扣减这类规则塞进 run()，它就必须跟着业务一起发版，任何一条业务线改动都要全量重启入口；不同服务对同一路径的诉求会挤进同一个过滤器，退回"改一处动全身"。更隐蔽的代价是可移植性：Provider 被迫依赖网关约定的头与上下文，换个网关（甚至换成 OpenResty）就跑不起来。判断标准很直接——凡是"换个业务域就要改"的代码，都不该出现在网关；网关只放"换个业务域也不变"的横切规则。

今天怎么做：Spring Cloud Gateway 与 Zuul 1 的 I/O 模型差别

今天怎么做：Zuul 1 已进入维护模式，现代替代是 Spring Cloud Gateway，构建在 WebFlux 之上，用 GlobalFilter 与链式调用（chain）取代 ZuulFilter 的四阶段。变的先是 I/O 模型：Zuul 1 是 Servlet 阻塞模型，每个在途请求占住一个容器线程，高并发下线程数与上下文切换就是天花板；Spring Cloud Gateway 基于事件循环与非阻塞 I/O，少量线程即可维持大量连接，更适合长连接与高扇出。其次是表达形式：四类型 + filterOrder 被统一的 GlobalFilter + 顺序接口取代，pre/route/post 的边界被"链前处理 / 转发 / 链后处理"重新表达，路由同样由服务发现解析成实例。不变的是职责边界：单一入口、路由到服务、横切前置、错误统一兜底、网关自身无状态可横向扩展——这些一条都没变，所以理解了 Zuul 的过滤器链，读 Gateway 的过滤链只是换一套名词。

选型判断也不该只看性能数字：如果存量过滤器全是 ZuulFilter、且流量远未到线程瓶颈，迁移成本可能高于收益；如果目标是长连接、高并发或要与响应式下游打通，WebFlux 模型才真正解决问题。把"换组件"当成"换职责"，是这次映射里最该避免的误读。原书当年推荐 Zuul 用于一般生产场景、高并发用 Gateway，理由也正是 I/O 模型的差别，而不是功能多寡。

容易误解的地方：

- 很多人以为网关只是"转发一下"，其实 Zuul 自身的大部分功能都由过滤器实现、连路由也是 route 过滤器，因为转发只是这条链上的一环。
- 很多人以为 filterOrder 越大越先执行，其实值越小优先级越高，因为同类型内按该值升序排列。
- 很多人以为 error 过滤器只是"记个日志"，其实它能把最终结果返回客户端，因为异常会改走 error 阶段而非原阶段收尾。
- 很多人以为 sensitiveHeaders 配了就会把头传给下游，其实它列的是"要被剥掉"的头，因为外发时不能泄露 Cookie / Authorization 这类会话凭证。

记住这一句：网关的价值是把横切逻辑收成一条链，风险是把全站流量压在一个必须永远在线、又必须无状态的进程上。

第9章　身份识别：Spring Security、JWT 与会话共享

一句话：认证归网关、鉴权归服务；无状态令牌换来扩展性，代价是主动失效能力。

为什么需要它：登录态原先放在单体的 HttpSession 内存里，拆成 8 个 Provider 之后，用户在 A 实例登录、下一个请求落到 B 实例就被判未登录；给 Nginx 开了粘滞会话（sticky session），结果一台实例重启，上面几千个会话一起丢失。改成 Redis 集中存 Session 后确实不丢了，但每个 Provider 每次请求都要回 Redis 取一次用户信息，网关和 Provider 还都被迫依赖同一份会话结构——只要 UAA 往 Session 里加一个字段，全线都要跟着改。更要命的是一次越权：网关验完 token 就把原始令牌透传下去，某个 Provider 直接信任了这个头，而该头在内网其实可以被伪造。这一章把三件事分开：认证（authentication，"你是谁"）与鉴权（authorization，"你能做什么"）的分界、Spring Security 在原理层怎么把凭证放进上下文、JWT 与分布式 Session 各自把状态放在哪、以及网关验签之后身份该以什么形式、在怎样的信任边界（trust boundary）上传给下游。

核心结构：

概念	作用
认证 authentication / 鉴权 authorization	"你是谁" / "你能做什么"，是过滤器链上两个不同环节
Spring Security 过滤器链	请求进业务前逐层拦截、从请求头组装凭证、决定放行
SecurityContext / SecurityContextHolder	线程级容器，持有本次请求的认证结果
Authentication	凭证/令牌：principal、credentials、authorities、isAuthenticated
AuthenticationManager / ProviderManager	认证入口；后者维护 provider 列表，找"支持该令牌"的那个
AuthenticationProvider	supports() 判定能否处理、authenticate() 执行校验
UserDetails / UserDetailsService / GrantedAuthority	数据源里的用户（密文密码、是否可用）、加载接口、权限集合
JWT：header.payload.signature	三段 Base64 + 签名，声明（claims）放在 payload
分布式 Session / Spring Session / Redis	把会话从 Web 容器剥离，交给外部存储共享
UAA 用户账号与认证服务	登录时校验密码、签发令牌
USER-ID 请求头	网关验签后只把用户标识透传给下游

关键设计：

```
[登录]
浏览器 --用户名/密码--> UAA(账号与认证) --校验--> 用户库
                              |
                              +--> 分布式 Session(Redis)：用户ID/角色/盐
                              +--> 签发 JWT：header.payload.signature
浏览器 <-- JWT(存 localStorage，放 Authorization / token 头) --+

[每次请求]
浏览器 --带 JWT--> Nginx --> Zuul(pre 过滤器 + Spring Security)
                                  | 取头 -> 组装 Authentication
                                  | AuthenticationManager 找 Provider
                                  | subject 取 Session ID -> Redis 取盐 -> 验签
                                  v 通过：SecurityContext.setAuthentication(...)
                              pre 过滤器 addZuulRequestHeader("USER-ID", ...)
                                  |   （不透传原始 token）
                                  v
                            Provider：按 USER-ID 鉴权 / 做业务，不重复认证

信任边界：网关之外不可信；网关到 Provider 是否可信，决定要不要再验一次
```

```
JWT 三段（示意）
header  {"typ":"JWT","alg":"HS256"}        -> Base64
payload {"sub":"...","exp":...,"iat":...}  -> Base64（明文，不放敏感信息）
signature = HMAC256( base64(header) + "." + base64(payload), salt )
验签：用同一 salt 重算签名比对 -> 一致才说明未被篡改
```

注释里的"为什么"：payload 只编码不加密，签名保证的是"没被改过"，不是"读不到"；所以密码、密钥绝不能放进去。

拆解：

认证与鉴权的分界，以及 Spring Security 的过滤器链

先把两个词钉死：认证（authentication）回答"你是谁"，产出的是一个已验证的身份；鉴权（authorization）回答"你能做什么"，拿这个身份去比对权限集合。二者在 Spring Security 里是不同环节——把凭证放对地方只是第一步，anyRequest().authenticated() 之外的权限判断才是第二步。对应到微服务：网关负责认证（token 是不是有效、是不是过期、是不是本人），下游服务负责鉴权（这个用户能不能改这条订单）。把鉴权也全压在网关上，网关就得懂每个服务的业务规则，又退回到"网关写业务逻辑"的反模式；反过来让每个 Provider 各自验一遍 token，则是重复验证，既降性能又拖累可移植性。

原理层只需要记住一条链和三个对象。请求进 Web 容器后先过一串过滤器，某个 OncePerRequestFilter 从 HTTP 头里取出 token、组装成 Authentication 令牌，交给 AuthenticationManager；它的实现 ProviderManager 维护一个 AuthenticationProvider 列表，用 supports() 找出能处理该令牌的提供者，再调 authenticate() 校验，成功后把结果写进 SecurityContextHolder 持有的 SecurityContext，失败则清上下文并交给失败处理器返回 403。认证通过后业务代码在任何地方都能从上下文取到当前身份，靠的就是这个线程级

容器；也正因为它是线程级的，跨线程异步编排时身份不会自动带过去，这是响应式链路里必须显式传递上下文的根因。

三个对象的职责要分清：Authentication 是"待验证或已验证的凭证"，含 principal（身份，可以是用户名或 UserDetails）、credentials（密码）、authorities（GrantedAuthority 权限集合）、isAuthenticated；UserDetails 是数据源里的用户，含密文密码、账号是否过期/锁定/可用；UserDetailsService.loadUserByUsername() 负责按名加载，PasswordEncoder 负责明文与密文匹配——原书特别点明这是"编码（Hash）"而非"加密"，因为不可逆，数据库泄露也看不到明文，忘记密码只能重置。DaoAuthenticationProvider 把"请求里的明文"和"数据源里的密文"用同一个 PasswordEncoder 对上，整个认证的信任根就落在这里；两处若用了不同编码器，匹配必然失败，这也是"配了却认证不过"的常见原因。原书还给出一个工程判断：建议只在网关启用 Spring Security，集群内部 Provider 不启用，要关闭自动配置，排除 SecurityAutoConfiguration 即可。

Session 在单体成立、在微服务成为负担

HTTP 本身无状态，服务端要认出"还是这个用户"，就得存一份会话并给客户端一个 Session ID。单体里这份状态放内存最自然，零网络开销、失效就是删掉对象。一旦横向扩展成多实例，麻烦来自三个方向：粘滞会话（sticky session）把用户绑到固定实例，实例一重启会话全丢，扩容还会导致分布不均；会话复制让每个实例都持有一份，节点越多同步写放大越严重，且复制窗口内各实例视图可能不一致；集中式 Session 把状态挪到 Redis（Spring Session 用 springSessionRepositoryFilter 把 HttpSession 换成 RedisSession，由 SessionRepository / RedisOperationsSessionRepository 读写），确实解决"丢"，但每次请求多一次网络 I/O，且网关与所有 Provider 被迫共享同一份会话结构，耦合度随节点数上升。

Spring Session 的设计值得单独记：它抽象出自己的 Session 接口、再用适配器适配 Servlet 的 HttpSession，是为了同时覆盖 HTTP、WebSocket、非 Web 等传输与多种存储；RedisSession 里 cached 做本地缓存优先读、delta 只跟踪变化的属性、saveDelta() 只持久化差异，都是在压那一次网络 I/O 的成本。但这些都只是"把负担挪走"，没有消除状态本身；只要状态还在服务端，"谁能改它、改了谁要跟着变"就始终是架构问题。

方案	状态存哪	失效能力	适用面
单体内存 Session	单进程内存	强：删对象即失效	单实例、无横向扩展
粘滞会话	绑定到某实例内存	中：实例宕机即丢	小规模、能容忍重登
会话复制	每个实例各一份	中：同步存在窗口	节点少、写不频繁
集中式 Redis Session	外部 Redis	强：删 key 即全局失效	节点多、要即时踢人
自包含 JWT	不存服务端（在令牌里）	弱：到期前无法主动作废	高扩展、可容忍短时无法注销

这张表的判断顺序是：先问"要不要即时踢人、改权限立刻生效"，要，就别选纯无状态；再问"节点规模与 I/O 预算"，大就别选复制；最后才在集中式与令牌之间权衡。原书那套 UAA 方案其实是混合态：JWT 的 subject 存的是 Session ID，网关验签时仍要回 Redis 取盐、查会话是否存在，甚至比对令牌字符串来拦重复登录——它用无状态外壳换扩展，却把失效能力留在了服务端。这也说明"无状态"不是非黑即白，而是把状态放在链路哪一段的取舍。

JWT 的三段、验签，以及无状态的收益与代价

JWT（JSON Web Token，RFC 7519）是一种凭证编码规范，用点号把三段连起来：header（typ、alg）声明类型与算法，payload 放声明（claims），signature 是把前两段的 Base64 用 header 指定算法加盐（salt）算出的签名。原书用 Algorithm.HMAC256 签发、JWT.require(...).build() 构造验证器再 verify(...) 演示验签；校验方用同一 salt 重算签名比对，一致才说明内容未被篡改。标准声明有 iss（签发人）、sub（主题）、aud（用户）、iat（签发时间）、exp（过期时间）、nbf（此前不可用），还能加 uid、user_name 这类私有声明。

收益来自"自包含"：网关验签不必回查用户库，Provider 不必共享会话，水平扩展时加实例不用同步任何状态，天然适合单点登录（SSO）与多网关。代价同样来自自包含，且每一条都要提前认账。第一，无法主动失效：令牌在 exp 之前一直有效，注销、改密、踢人都不能立刻生效，要补黑名单或退回集中式 Session——这正是上一节对比表里"弱"的来源。第二，令牌体积：私有声明越塞越多，每个请求都要多带一段 Base64，且 payload 是编码后的明文，只防篡改不防读取，绝不能放密码、密钥、敏感信息。第三，密钥轮换（key rotation）：salt 一换，旧签名的令牌全部验不过，必须设计并行验证旧密钥的过渡期，否则等于全量强制下线；原书把用户密码密文当盐，好处是改密即自然作废旧令牌，坏处是密钥与用户数据绑死、轮换粒度很粗。第四，过期时间与刷新令牌（refresh token）：exp 设长了风险高、设短了用户反复登录，所以配套 oauth 客户端配置里才有 access_token_validity 与 refresh_token_validity 两个时长，用短时访问令牌换安全、用长时刷新令牌换体验，刷新令牌本身也要能被服务端作废。

把这些代价摆在一起，判断标准就清楚了：JWT 不是"比 Session 先进"，而是拿"主动失效能力"换"免状态同步"。业务要求即时注销、权限实时生效时，纯无状态是错误选择；能容忍一个短过期窗口、又需要多网关水平扩展时，它才划算。

信任边界与今天怎么做：身份怎么传给下游、内部调用要不要带身份

网关验签通过之后，身份怎么到 Provider 手里，是这一章最容易被忽略的设计点。原书列了三种：透传整个 JWT、只透传 subject 里的 Session ID、或把会话里缓存的用户 ID 取出来加一个 "USER-ID" 头。前两种都要求 Provider 与网关共享同一套会话/密钥体系，而负责认证的网关未必总是 Zuul——换成性能更高的 OpenResty 甚至 Kong 时，共享 Session 的难度陡增；原书因此推荐第三种，并用一个 pre 类型的过滤器在 run() 里 addZuulRequestHeader 完成。这个选择的本质不是"哪种头更短"，而是把下游从"必须懂令牌格式"里解耦出来：Provider 只认一个用户标识，不认签发算法、不认盐、不认会话结构。

信任边界就在这里：网关之外是不可信的公网，必须验签；网关到 Provider 这一段是否可信，决定"下游还要不要再验一次"。请求头透传的本质是"下游信任网关已经验过"，一旦这个头在内网可被绕过直连伪造，信任就崩了——所以要么在网络层保证 Provider 只能经网关到达，要么让下游自行验签（代价是每个服务都绑定密钥、回到重复校验）。两种不是对错，是信任边界画在哪、以及愿不愿意为它付网络隔离的成本。把 USER-ID 这种"可被任意构造的明文标识"当成鉴权依据，前提永远是"这个头只可能来自可信的网关"。

随之而来的是服务提供者之间的会话共享关系。原书对比了两种：全局共享（所有网关与 Provider 共用一个分布式 Session，节点少时可行，规模一大 I/O 成瓶颈且耦合极高）、局部按需共享（网关与 Provider 不再传 Session ID，改传 User ID，需要会话的服务再按 User ID 去 Redis 反查绑定关系）。后者给了那个关键判断：内部调用是否还需要带用户身份，取决于业务——只有当下游要按用户做鉴权、审计或数据隔离时才需要透传；纯内部的服务间调用、定时任务、异步消息消费不带用户身份反而更干净，因为带身份等于重新把用户上下文耦合进每一跳。判断标准一句话：谁要按"人"做决策，谁才需要身份。

今天怎么做：把认证从各服务里彻底收走，交给 OAuth2 / OIDC 的授权服务器统一签发，网关只做令牌校验与路由，下游作为资源服务器按 scope 与权限鉴权；对外用短时访问令牌 + 刷新令牌缓解"无法主动失效"，对内

走向零信任（zero trust）——服务间调用也带可验证的服务身份，而不是默认信任内网；要即时踢人，就把失效判断收敛到一个轻量黑名单或足够短的 exp 上。变的是令牌格式与校验位置，不变的是那条边界：认证在入口一次做完，身份以最小必要形式向下游传递，鉴权留在离业务最近的地方。

容易误解的地方：

- 很多人以为认证通过就等于有权限，其实认证只回答"你是谁"、鉴权另由 authorities 决定"你能做什么"，因为二者是过滤器链上不同环节。
- 很多人以为 JWT 的 payload 是加密的，其实它只是 Base64 编码、等同明文，签名只防篡改不防读取，所以不能放密码等敏感信息。
- 很多人以为用了 JWT 就能随时让令牌失效，其实自包含令牌在到期前服务端无法主动作废，要失效就得补黑名单或退回集中式 Session，因为验签不查库。
- 很多人以为网关把原始 token 原样透传最省事，其实更稳的是只透传一个下游能自证的用户标识，因为透传 token 等于让所有 Provider 绑定同一套密钥与会话体系。

记住这一句：认证解决"是谁"，鉴权解决"能做什么"；把状态拿掉换来的扩展能力，要靠短时令牌、信任边界和最小透传补回来。

第10章　Nginx：进程模型、location 与反向代理

一句话：Nginx 靠 master/worker 多进程 + 异步非阻塞事件驱动扛高并发；`location` 匹配优先级与 `proxy_pass` 的路径拼接是最高频的"配了不生效"现场。

为什么需要它：Spring Cloud 体系里 Nginx/OpenResty 站在最外缘，负责连接复用、静态资源、TLS、第一道限流和边缘 Lua 逻辑。现场：你把 `worker_connections` 调很大，却扛不住预期的并发——因为总连接数还受 `worker_processes` 和系统文件描述符限制。现场二：配了一条 `location /api` 规则，结果请求走了另一条——匹配优先级没搞清。现场三：`proxy_pass` 末尾带不带 `/` 行为完全不同，导致路径拼接错误、后端 404。现场四：写了 `rewrite` 规则却陷入死循环，返回 500。这一章讲清 Nginx 为什么能扛并发，以及配置里最坑的三处。

核心结构：

概念	作用
master 进程	读取配置、绑定端口、fork/管理 worker，自身不处理请求
worker 进程	实际处理连接，数量通常 = CPU 核数，单进程异步非阻塞
事件驱动 (epoll)	一个 worker 用一个事件循环管大量连接，避免每连接一线程
配置层级	main / events / http / server / location，内层继承外层
`location`	按 URI 匹配请求，决定走哪段处理规则
`upstream`	定义一组后端及负载策略，供 `proxy_pass` 引用
`proxy_pass`	反向代理到后端，路径拼接受末尾 `/` 影响

关键设计：

```
# 示意：常见结构（非生产全量）
worker_processes  auto;           # 通常 = CPU 核数
events { worker_connections 10240; }   # 单 worker 最大连接

http {
  upstream order_svc {            # 后端组 + 负载策略
    least_conn;                  # 或 round-robin / ip_hash
    server 10.0.1.7:8080; server 10.0.1.8:8080;
  }
  server {
    listen 80;
    location = /health { ... }                 # 精确匹配，优先级最高
    location ^~ /static/ { root /data; }       # 前缀匹配，命中即停（不查正则）
    location ~ \.php$ { ... }                  # 正则匹配（区分大小写）
    location /api/ {                           # 普通前缀
      proxy_pass http://order_svc/;            # 末尾有 /：把 /api/ 去掉再拼
      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
    }
  }
}
```

```
  location 匹配优先级（高→低）
    1）= 精确匹配
    2）^~ 前缀命中即停（不再查正则）
    3）正则 ~ / ~*（按配置顺序，第一个命中）
    4）普通前缀（最长匹配）
    5）/
```

拆解：

10.1 为什么 Nginx 能扛高并发

传统"每连接一进程/线程"模型在 C10K 问题上会因上下文切换和内存爆炸而崩。Nginx 用 master/worker：**master 只管进程与端口**，真正干活的是多个 worker；每个 worker 是**单线程事件循环**，借助 `epoll`（Linux）等机制在一个线程里同时驱动成千上万连接——连接空闲时不占线程，只在有事件时回调处理。于是"连接数"与"线程数"解耦，少量线程即可服务海量连接。这点和第 6 章"线程池隔离"思路相反：Nginx 靠"不阻塞"而非"多隔离"来扩展。

10.2 配置层级与继承

配置是嵌套的：`main`（全局）→ `events`（连接模型）→ `http`（协议级）→ `server`（虚拟主机）→ `location`（URI 规则）。内层继承外层，可被覆盖。理解层级，就知道"超时配在 http 还是 server"会影响作用范围；也知道为什么有些指令只能出现在特定层级（如 `worker_connections` 在 `events`）。

`worker_connections × worker_processes` 是理论最大连接，但还要受系统 `ulimit -n` 和事件模块限制。

10.3 location 匹配优先级

这是"规则没生效"的头号原因。顺序：`=` **精确 >** `^~` **前缀(停) > 正则(顺序首个) > 普通前缀(最长) >** `/`。常见坑：写了 `location /api` 普通前缀，却被更靠前的正则或 `^~` 抢走；或以为"正则一定优先于普通前缀"——其实 `^~` 和普通前缀都先于正则。排查时按上表逐条比对，而不是凭直觉。

10.4 proxy_pass 路径拼接与 rewrite

`proxy_pass` 末尾带 `/` 与不带 `/` 天差地别：带 `/` 时，匹配的 `location` 前缀会从转发路径中去掉；不带 `/` 时，完整 URI 原样转发。这直接决定后端收到的路径，拼错就 404。配合 `proxy_set_header` 把真实客户端 IP/Host 透传给后端（否则后端只看到 Nginx 的地址）。`rewrite` 用正则改写 URI，`last`（内部重新匹配 location）/ `break`（停止重写不再匹配）/ `redirect` / `permanent`（外部跳转）语义不同；用 `last` 改写后又命中自身会死循环返回 500，需谨慎。`upstream` 还支持 `least_conn` / `ip_hash` 等策略与基础健康检查。

10.5 OpenResty/Lua 的插入点

OpenResty 让 Nginx 在请求处理的不同阶段执行 Lua：`access_by_lua`（访问控制/限流）、`content_by_lua`（直接产出响应）、`header_filter_by_lua`（改响应头）等。把"读缓存、鉴权、限流"放到 Nginx 层，可省去一次到后端的跳转、降低延迟。代价是逻辑分散在边缘、难以单元测试，且与业务代码两套实现容易不一致——只适合"无状态、高复用"的边缘逻辑，别把业务塞进来（呼应第 8 章网关同理）。

今天怎么做：云原生下，Nginx 常以 **Ingress Controller** 形式存在，或用 **Gateway API** 替代手写 location；OpenResty 仍适用于需要边缘 Lua 的场景（如自定义限流、灰度）。但"异步事件驱动、location 优先级、proxy_pass 拼接、边缘逻辑只放无状态"这些不变。

容易误解的地方：

1. 以为"`worker_connections` 调大就高并发"——总连接受 `worker_processes × worker_connections` 与系统 fd 上限双重约束。
2. 以为"正则一定优先于普通前缀"——`^~` 和普通前缀先于正则，顺序不是直觉那样。
3. 以为"`proxy_pass` 带不带 `/` 无所谓"——决定后端收到的路径，拼错即 404。
4. 以为"Lua 逻辑随便放 Nginx 层"——边缘逻辑难测试、易与业务重复，只放无状态高复用逻辑。

记住这一句：Nginx 靠 master/worker + 事件驱动扛连接，靠 location 优先级和 proxy_pass 拼接定路由——这两处弄错，就是"配了不生效"的重灾区。

第11章　限流：计数器、漏桶、令牌桶与分布式实现

一句话：限流是在系统入口拒绝超额流量，熔断（第 6 章）是在下游故障时止损，背压（第 7 章）是链路内部协商速率——三者位置与目的都不同。

为什么需要它：限流解决的是"高并发大流量下，保护接口/系统不被压垮"。现场：某接口能扛 1 万 QPS，来了 2 万请求，不限制就全员超时、数据库被打穿。现场二：你给每个实例配了 1000 QPS 限流，以为全局安全，结果 10 个实例 = 1 万 QPS 被放行，数据库照样崩——因为"每实例限流 ≠ 全局限流"。现场三：用固定窗口计数器，结果在窗口边界瞬间涌入两倍流量仍被放行——窗口边界突刺。现场四：Nginx 配了 `limit_req burst=100`，却还是被拒——没理解 `nodelay` 语义。这一章把三种算法讲透，并区分单机和分布式限流。

核心结构：

算法	是否允许突发	输出是否平滑	实现代价	适用面
固定窗口计数器	边界有突刺	否	极低	粗粒度、容忍突刺
滑动窗口/滑动日志	较平滑	较平滑	中	统计更准
漏桶 leaky bucket	否（恒定速率）	是	低	需平滑输出、削峰
令牌桶 token bucket	是（桶容量内）	较平滑	低	允许适度突发（最常见）

关键设计：

```
三种算法示意

计数器(固定窗口):   [0,1s) 计数≤N，超了拒绝；窗口边界处可能 2N 涌入
漏桶:               请求进桶(固定容量)，以固定速率流出处理；桶满则丢弃
                      → 输出速率恒定，能削峰，不允许突发
令牌桶:             以固定速率往桶里放令牌(桶有上限)；请求来取令牌，无令牌则拒绝
                      → 桶里有存量时允许突发，长期速率=填充速率
```

```
分布式限流的难点
单实例："每实例限 N" → 总容量 = N × 实例数  ≠ 你想要的全局 N
分布式：必须有一个【全局原子计数器】
    方案：Redis INCR + EXPIRE（或 Lua 脚本保证原子）
    注意：多实例各自 INCR 同一 key，靠原子自增 + 统一过期窗口
    坑：时钟漂移、key 过期与统计窗口对齐、Redis 本身可用性
```

拆解：

11.1 限流与"熔断/背压"的边界

限流在**入口**拦截超额流量，保护的是"整个系统不被超额请求冲垮"；熔断在**下游故障**时切断对失败依赖的调用（第 6 章）；背压是**链路内部**上下游之间的速率协商（第 7 章）。三者互补：入口限流挡掉大部分超额，内部背压平滑生产者消费者差，熔断在依赖恶化时快速失败。把限流当成熔断用（或反过来）都会错位——例如只做熔断不做限流，入口流量照样把线程池打满。

11.2 三种算法及其取舍

- **固定窗口计数器**：在固定时间窗内计数，超限拒绝。实现最简单，但窗口边界会出现"上一窗末尾 + 下一窗开头"的 2 倍突刺（双窗叠加）。适合对突刺不敏感的粗粒度场景。
- **漏桶**：请求进入固定容量桶，以恒定速率流出被处理；桶满丢弃。输出速率绝对平滑、能削峰，但**不允许突发**——即使系统空闲，也得按固定速率放请求，可能浪费容量。
- **令牌桶**：以固定速率向桶放令牌（桶有上限），请求消费令牌，无令牌则拒绝。桶有存量时**允许一定程度的突发**，长期平均速率等于令牌填充速率。这是最常用算法，因为真实流量本就有突发，令牌桶对突发更友好。

11.3 单机限流 ≠ 分布式限流

多实例部署时，"每台限 1000 QPS"实际放行的是 1000 × 实例数。要限"全局 1000 QPS"，必须有一个**跨实例的全局计数**。常用 Redis：`INCR` 自增 + `EXPIRE` 设窗口，或用一段 **Lua 脚本**把"自增 + 判断 + 重置"做成原子操作，避免 `INCR` 与 `EXPIRE` 之间出现竞态导致窗口错乱。还要注意：Redis 的可用性与时钟一致性、key 的过期与统计窗口对齐、以及在 Redis 不可用时"放开还是收紧"的兜底策略（通常收紧更稳妥，但要避免误杀）。

11.4 Nginx 限流与阈值设定

Nginx 提供 `limit_req_zone` + `limit_req`（基于漏桶/令牌思想的请求限速，`burst` 允许排队突发，`nodelay` 表示突发不延迟直接放行）、`limit_conn`（并发连接数限制）。`burst` 配了还被拒，往往是没加 `nodelay` 或 `zone` 共享范围不对。阈值怎么定：**先做容量压测找到依赖最弱一环的承载上限**，再按"用户/接口/维度"分级限流——对核心用户宽松、对爬虫严格，对写接口更紧。拒绝后要返回明确状态码（如 429）并给客户端退避建议，而不是静默丢请求。

今天怎么做：**Sentinel**（流量控制 + 熔断降级一体，支持热点参数限流、集群限流）、**Resilience4j RateLimiter**、网关层统一限流都是主流；云上有托管的 WAF/API 网关限流。但"算法选择（突发 vs 平滑）、全局原子计数、阈值按最弱依赖定、分级限流"这些判断不变。

容易误解的地方：

1. 以为"每实例限 N 就全局安全"——总容量是 N×实例数，必须分布式计数才能限全局。
2. 以为"漏桶和令牌桶差不多"——漏桶恒定输出不允许突发，令牌桶允许桶内突发，语义相反。
3. 以为"固定窗口计数器最准"——窗口边界有 2 倍突刺，统计上最不平滑。
4. 以为"Nginx burst 配了大就不会被拒"——缺 `nodelay` 时突发仍会被延迟或拒。

记住这一句：限流是入口拒绝，熔断是下游止损，背压是内部协商；分布式限流必须靠全局原子计数，每实例各限各的等于没限。

第12章　秒杀全链路：把前 11 章串成一条线

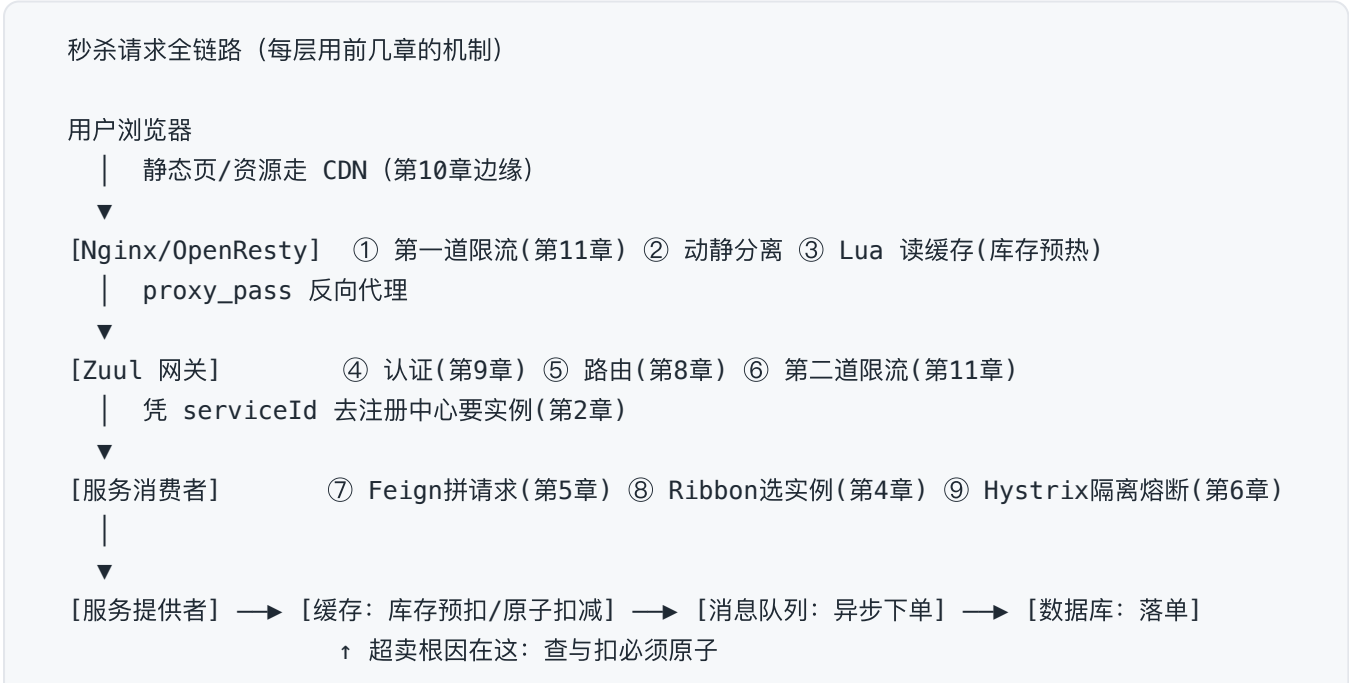
一句话：秒杀把前面 11 章的机制压到同一条链路上——就近拦截、动静分离、原子扣库存、异步削峰，每一层用的都是前几章讲过的判断标准。

为什么需要它：秒杀是微服务和中间件能力的综合考场：瞬时流量是平时的几十上百倍，且 95% 是无效请求（库存只有 100 件，却来了 10 万请求）。任何一个环节没拦住，压力都会传导到数据库，造成超卖、雪崩、连接耗尽。现场：活动一开始数据库 CPU 直接打满——因为所有请求都透到了查库存/扣库存。现场二：卖出 120 件，库存只有 100——"查库存"和"扣库存"不是原子操作，并发下多扣。现场三：同一用户下了两单——异步链路里"下单成功"和"最终成交"被当成同一时刻，重复消费没去重。这一章用秒杀把全书串起来，并给出四个关键判断。

核心结构：

链路层	用了哪章的机制	挡住什么
CDN/静态化	第 10 章边缘	静态资源流量不进后端
Nginx/OpenResty	第 10/11 章	第一道限流、动静分离、Lua 读缓存
网关 Zuul	第 8/11 章	认证、路由、第二道限流
消费者 Feign/Ribbon/Hystrix	第 4/5/6 章	选实例、隔离熔断、降级
注册/配置中心	第 2/3 章	实例与配置来源
缓存/消息/数据库	第 12 章特有	库存预扣、异步下单、最终一致

关键设计：



拆解：

12.1 流量要在离用户最近处被挡掉

无效的 10 万请求，越早拒绝成本越低：在 CDN/Nginx 层挡掉，连应用进程都不进；放到网关层挡，已经消耗了连接与路由；放到数据库层挡，系统早已过载。所以**多层限流是叠加而非冗余**——每一层按自己的职责挡一部分：Nginx 挡边缘突发，网关挡服务级超额，应用内挡依赖过载。这正呼应第 11 章"限流是入口拒绝"，以及第 8 章"内外网关分层"。

12.2 读多写少：静态化、缓存、库存预扣各管一段

秒杀 95% 是读（看商品、看库存）和无效写（已售罄还点）。三类手段解决不同问题：**静态化/CDN** 让读不进后端；**缓存** 承接"查库存"这类高频读；**库存预扣** 把"扣库存"从数据库提到缓存/原子操作。注意三者边界：缓存和数据库的一致性只在"最终"成立，不能假设缓存永远等于数据库；"预扣"只是把竞争从数据库上移到更轻的量级存储，但仍是并发点，必须保证原子。

12.3 超卖根因：查与扣不是原子

超卖的本质是"判断有库存"和"扣减库存"两个动作之间有空隙，并发下多个请求都读到"还有 1 件"，各自扣成 -1。解决不是"加分布式锁了事"（锁粒度大会压垮性能），而是**把查与扣做成原子操作**：用原子自减（`DECR` 类原语，返回值 ≤0 即售罄）、或数据库 `UPDATE ... SET stock=stock-1 WHERE stock>0` 的条件更新、或在缓存层用 Lua 脚本把"判断+扣减"串行化。原子扣减之后，再对通过校验的请求做后续处理——这样锁竞争的只有"扣库存"这一个极小临界区。

12.4 异步化：下单成功 ≠ 最终成交

秒杀不能同步完成"扣库存+创建订单+支付"，否则数据库在峰值被冲垮。正确做法是：原子扣库存通过后，把"创建订单"作为消息发到队列，**立即返回"排队中"**。后台消费者异步落单。这把"下单受理"和"最终成交"拆成两个时刻——于是第 6 章讲的状态机、第 4 章讲的消息重复、幂等键、以及"结果未知时主动查询/补单"全部重新出现：消息可能丢失（需确认机制）、可能重复消费（需幂等）、订单可能处于"处理中"（需状态收敛）。这和第 1 章"分层、每层有失败边界"、支付书里的"状态机"是同一套思想。

今天怎么做：云原生下可用 **API 网关限流 + Sentinel 集群限流 + Redis 原子扣库存 + MQ 异步削峰 + 分库分表**实现同样链路；Service Mesh 还能把重试/熔断下沉到 sidecar。但"就近拦截、动静分离、原子扣库存、异步削峰、幂等去重"这套判断标准不变——技术换了，考题一样。

容易误解的地方：

1. 以为"加分布式锁就能防超卖"——锁错位置/粒度会压垮性能，原子扣减才是正解。
2. 以为"下单返回成功就是成交了"——异步链路里两者是两个时刻，中间靠状态机与幂等收敛。
3. 以为"一层限流就够了"——越靠近用户拦截成本越低，多层是职责叠加。
4. 以为"缓存里的库存等于数据库"——只保证最终一致，不能假设实时相等。

记住这一句：秒杀没有新概念，只是把前 11 章的判断标准压到一条链路上——就近拦、原子扣、异步削峰、幂等兜底，技术换了考题不变。

结语：微服务不是组件的堆砌

把 Nginx、网关、注册中心、配置中心、Feign、Hystrix、限流一个个用起来，和真正"设计"一套微服务，是两件事。前者是照着文档拼组件，后者是清楚每一层在替你挡什么、放过了什么、又会引入什么新的失败模式。

12 章串起来只有一条主线：**把"一次请求"在各层的职责切干净，让每一层只解决自己那段问题，并且每一层都有明确的失败边界**。Nginx 挡边缘流量，网关做服务级横切，Feign/Ribbon 解决"怎么把调用发出去、发给谁"，Hystrix 解决"下游不行时上游怎么自保"，注册/配置中心解决"实例和配置从哪来"，限流解决"入口超额流量怎么拒"。哪一层的职责被挪到另一层，哪一层就会同时失去自己的能力和对方的可靠性。

技术栈会换：Eureka 换成 Nacos，Hystrix 换成 Sentinel，Zuul 换成 Gateway。但"选机器的决策发生在哪一侧""熔断挡的是雪崩传导""限流是入口拒绝而不是内部反馈""库存扣减必须是原子操作"——这些判断标准不会变。读这本书的真正收获，是下次你面对一个微服务故障或设计取舍时，能立刻说出"这是第几章的问题"。

把书读薄：一张判断清单

- 请求从浏览器到数据库，列出它穿过的每一层；任何一层出事，先判断它本该解决哪段问题。
- "服务还在被调用" → 先看注册发现的 AP 语义和本地缓存，再看负载均衡是否绕过了发现。
- "配了不生效" → 配置被谁加载（bootstrap 还是 application）、被谁覆盖、在哪个时刻被读取。
- "整条链路卡死/雪崩" → 是超时没配好，还是没有隔离/熔断让一个慢下游耗尽上游资源。
- "限流没用" → 先问限的是单机还是全局；分布式限流必须靠原子计数，不能每实例各限各的。
- "超卖/重复下单" → 检查"查库存"和"扣库存"是否原子、下单和成交是否被当成了同一时刻。
- 任何横切逻辑（鉴权、日志、限流）放错层（尤其在网关写业务、在应用做边缘限流），迟早要还债。

附录：三张速查表

表 A：组件 / 职责边界 / 现代替代

组件（原书）	职责	现代替代
Eureka	服务注册与发现（AP）	Nacos、Consul
Ribbon	客户端负载均衡	Spring Cloud LoadBalancer
Feign	声明式 RPC 客户端（动态代理）	OpenFeign（仍用）
Hystrix	隔离/熔断/降级	Resilience4j、Sentinel
Zuul	内部网关（Servlet 阻塞）	Spring Cloud Gateway
Config	配置中心	Nacos Config、Apollo
Nginx/OpenResty	边缘反向代理 + Lua 扩展	Nginx Ingress、Gateway API

表 B：配置参数 / 归属层 / 不生效的常见原因

参数	归属层	不生效的常见原因
`eureka.client.*`	注册发现	客户端缓存未刷新、自我保护触发
`spring.cloud.config.*`	配置中心	写在 application 而非 bootstrap、仓库路径错误
Feign `connectTimeout/readTimeout`	调用客户端	被 Ribbon/Hystrix 超时先触发、未显式配置
Ribbon `NFLoadBalancer*Rule*`	负载均衡	被 Feign 配置覆盖、规则类未注册
Hystrix `execution.isolation.*`	熔断隔离	依赖隔离策略、线程池维度配错
Nginx `location` / `proxy_pass`	边缘代理	匹配优先级、结尾 `/` 拼接规则

表 C：故障现象 / 首选章节 / 根因类型

现象	首选章节	根因类型
调用到已下线实例	2、4	发现 AP + 本地缓存
配置改了不生效	3	加载阶段 / 覆盖 / 读取时机
超时配了也卡死	5、6	多套超时 + 无隔离
雪崩	6	无舱壁/熔断
网关膨胀难维护	8	横切逻辑放错层

现象	首选章节	根因类型
会话/令牌问题	9	认证鉴权边界
Nginx 规则不生效	10	匹配优先级/路径拼接
限流形同虚设	11	单机 vs 分布式
超卖/重复下单	12	原子性/异步化

最后一句

微服务的复杂度，从来不是组件多，而是每一层都在把"别人的失败"变成"自己的事"——把职责切干净、把失败边界标清楚，书就变薄了。